

具体数学选讲

qwaszx

2026 年 4 月 6 日

从本书能学到的东西：

- 一些方法和技巧
- 大师是如何思考问题的
- “数学成熟度”

本书直到 5.4 为止都是对 OI 非常有用的内容。
再往后的内容与 OI 关联不大，但读了也能有很大收获。

另一本书：《离散数学及其应用》

强烈建议把这本书的前两章看一看，讲了很多重要且基础的数学语言。
后面也都是和 OI 非常相关的内容，相对比较基础，有空的话可以看看

每周上课前需要大家自行阅读《具体数学》的对应章节，并完成习题。上课时我们只会快速地过一遍教材并强调重点。

做习题的时候尽量不要翻前面的正文，以加深记忆。

其实建议大家习题全做的，只不过为了减轻大家的工作量只选了一部分有代表性的让大家写下来。

作业要求

我们的期望任务量是每周 8-10 个小时 (阅读 + 做题), 大家在填写每周的任务反馈时可以参考这个值

作业中遇到努力思考但仍然不会的题目时, 有以下几种解决办法:

- 翻到附录 A 查看答案
- 询问同学
- 询问我: 你需要表达清楚卡住的点 (比如已经尝试过什么、到哪里做不下去了、或者干脆一点头绪没有等等), 我会努力提供一个合适的提示 (但是不一定能及时回复)
- 询问 AI: 你可以让 AI 模仿上一条或者答案或者别的什么, 但是正确性需要自行甄别

总之, 希望大家像对待 OI 题一样对待数学习题。

提交作业前, 建议大家先对一下附录 A 的答案并让 AI 批改一遍。参考提示词:

- 帮我批改一下这份作业, 题目是具体数学第 x 章后面的对应习题。(后附作业内容)

第一章

- 递归式的概念
- 数学归纳法
- 如何写出递归式（简单的例子）
- 一种解递推式的方法

需要写下来的：1 3 5 8 9 12 14 16

练习：写递归式

其实写 dp 的时候会经常练。

- ① 骨牌覆盖：用 1×2 和 2×1 的骨牌覆盖一个 $2 \times n$ 的网格，求方案数
- ② 骨牌覆盖 2：用 1×2 和 2×1 的骨牌覆盖一个 $3 \times n$ 的网格，求方案数
- ③ 长为 n 的 01 串，要求不能出现连续的三个 1，求方案数
- ④ 长为 n 的 01 串，要求不能出现连续的三个 1 或者连续的两个 0，求方案数
- ⑤ 用 m 种颜色给 $1 \times n$ 的网格涂色，要求相邻格子颜色不相同
- ⑥ 长为 $2n$ 的合法括号序列的数量
- ⑦ 从 $(0,0)$ 走到 (n,n) ，每步可以向上或向右走，不能跨过对角线的方案数
- ⑧ n 选 k 排列数
- ⑨ n 选 k 组合数
- ⑩ 把 n 写成若干单调不降正整数之和的方案数 (hint: 你也许需要添加一维)
- ⑪ n 个人围成一圈，选若干个人，要求不能相邻，求方案数
- ⑫ n 个数的排列，要求 $a_i \neq i$ ，求方案数

你还能提出其他的问题吗？你能推广上面的某些问题吗？（无需会做）

① TODO

- 骨牌覆盖：用 1×2 和 2×1 的骨牌覆盖一个 $n \times m$ 的网格，求方案数
- 长为 n 、字符集大小为 m 的字符串，要求某些子串不能出现，求方案数
- 从 $(0, 0)$ 走到 (n, m) ，每步可选一个集合内的一种，另有对不能经过的位置的限制，求方案数
- 把 n 写成若干单调不降正整数之和的方案数，额外要求：严格递增/数都不超过 k /数都至少是 k /必须是奇数/...
- n 个数的排列，要求恰好 k 个位置 $a_i \neq i$ ，求方案数

解递归式

我们有两种目的：得到一个**封闭形式**，或者**快速计算**。

对于只有一个递归项的递推式 $f_n = a_n f_{p(n)} + g(n)$ ，可以考虑展开变成求和：

$$f(n) = g(n) + a_n g(p(n)) + a_n a_{p(n)} g(p(p(n))) + \cdots$$

对于形如 $f_n = \sum_{i=1}^k a_i f_{n-i} + r^n P(n)$ （其中 $P(n)$ 是多项式）的递推，可以参考**常系数线性递推理论**

对于多个相互依赖的线性递推，可以考虑生成函数。不过，我们还可以写成**矩阵形式**直接算。

对于二维的递推，可以尝试的做法是对其中一维建立生成函数，然后转为生成函数的一维递推。但这通常也很困难。

对于分式类型的递推（如 $f_n = \frac{1}{f_{n-1}+1}$ ），可以尝试将 f_n 写为 p_n/q_n ，然后导出对应的递推

其他的递推往往很难解。

展开递归式：举例

$$J(n) = 3J(\lfloor n/2 \rfloor) + \gamma n + a_{n \bmod 2}$$

“成套方法”

事实上，下面的思想往往更有用：在处理非齐次递推（如 $f_{n,m} = f_{n-1,m} + f_{n,m-1} + g(n,m)$ ）时，可以转而考虑每个非齐次项 $g(i,j)$ 和初值项 $f_{0,\cdot}, f_{\cdot,0}$ 对最终的 $f_{n,m}$ 的贡献系数。

“算贡献”的思想对解决其他问题常常也有帮助。

- 熟悉 $\sum$ 记号 (艾弗森括号、条件型下标)
- 扰动法
- 二重求和: 对称式
- 熟悉一些化简和式的方法
- 下降幂

第二章习题

需要写下来的：2 3 4 6 13 14 19 20 21 22 25 32; 7 9 10 11 17 23 24 27 29 31; 35* 36*

两周内完成，其中第一周至少要完成第一个分号前的习题（对应教材 2.1-2.5）。另外补充一些 OI 练习题：洛谷 P5686 P4948 P4091 U77198

化简和式的一些指导思想

- 尽量使用简单的求和条件
- 把无关项用分配律提出到求和号外
- 交换求和号常常是有用的

含有求和的递归

例: $s_n = \sum_{i=1}^n f_i$, f_n 是 Fibonacci 数列

邻项相减

本题还有包括但不限于扰动法的各种推导方式

利用对称去掉偏序

在三角求和的例子中，我们利用对称性把 $i \leq j$ 的求和变成无限制的求和以及 $i = j$ 的求和。这是常用的技巧。例如：

$$\sum_{1 \leq i \leq j \leq n} |x_i - x_j|.$$

异或粽子

另外，这个技巧可以推广为 Burnside 引理。

$$\sum_{1 \leq i_1 \leq i_2 \leq \dots \leq i_m \leq n} 1$$

是多少？

我们有很多方式来推导它：找规律后归纳、组合意义、有限微积分、写递归式，等等

找规律

打表发现答案是 $\binom{n+m-1}{m}$ 。

证明：按 $n + m$ 归纳：

$$\begin{aligned} & \sum_{1 \leq i_1 \leq \dots \leq i_m \leq n} 1 \\ &= \sum_{1 \leq i_1 \leq \dots \leq i_m < n} 1 + \sum_{1 \leq i_1 \leq \dots \leq i_{m-1} \leq i_m = n} 1 \\ &= \binom{(n-1) + m - 1}{m} + \binom{n + (m-1) - 1}{m-1} \\ &= \binom{n + m - 1}{m} \end{aligned}$$

- 对每个 $t = 1, 2, \dots, n$, 考虑有多少个求和指标等于 t 。这样相当于把 m 个球放到 n 个可空的桶里
- 或者, 考虑 $[1, i_1], (i_1, i_2], (i_2, i_3], \dots, (i_{m-1}, i_m], (i_m, n]$ 这 $m + 1$ 个桶, 其中第一个桶非空, 其他可空。
- 再或者, 每个 $1 \leq i_1 \leq i_2 \leq \dots \leq i_m \leq n$ 一一对应一条路径 $(1, 1) \rightarrow (1, i_1) \rightarrow (2, i_1) \rightarrow (2, i_2) \rightarrow (3, i_2) \rightarrow (3, i_3) \rightarrow \dots \rightarrow (m, i_m) \rightarrow (m + 1, i_m) \rightarrow (m + 1, n)$ 的路径, 其中 $(t, i_t) \rightarrow (t + 1, i_t)$ 是一个向右的步, $(t, x) \rightarrow (t, x + 1)$ 是一个向上的步。那么相当于从 $(1, 1)$ 出发向右向上走到 $(m + 1, n)$

$$\begin{aligned}
 \sum_{1 \leq i_1 \leq \dots \leq i_m \leq n} 1 &= \sum_{i_m=1}^n \sum_{i_{m-1}=1}^{i_m} \dots \sum_{i_1=1}^{i_2} 1 = \sum_{i_m=1}^n \sum_{i_{m-1}=1}^{i_m} \dots \sum_{i_2=1}^{i_3} i_2 \\
 &= \sum_{i_m=1}^n \sum_{i_{m-1}=1}^{i_m} \dots \sum_{i_3=1}^{i_4} \frac{i_2}{2} \\
 &= \sum_{i_m=1}^n \sum_{i_{m-1}=1}^{i_m} \dots \sum_{i_4=1}^{i_5} \frac{i_3}{2 \cdot 3} \\
 &\vdots \\
 &= \sum_{i_m=1}^n \frac{i_m^{\overline{m-1}}}{(m-1)!} = \frac{n^{\overline{m}}}{m!} = \binom{n+m-1}{m}
 \end{aligned}$$

$$\begin{aligned} f_{m,n} &= \sum_{1 \leq i_1 \leq \dots \leq i_m \leq n} 1 \\ &= \sum_{i_m=1}^n \sum_{1 \leq i_1 \leq \dots \leq i_{m-1} \leq i_m} 1 \\ &= \sum_{i_m=1}^n f_{m-1,n} \end{aligned}$$

差分得到 $f_{m,n} - f_{m,n-1} = f_{m-1,n}$ 。

或者，在一开始就考虑

$$\begin{aligned} f_{m,n} &= \sum_{1 \leq i_1 \leq \dots \leq i_m \leq n} 1 \\ &= \sum_{i_m=1}^n \sum_{1 \leq i_1 \leq \dots \leq i_{m-1} \leq i_m < n} 1 + \sum_{i_m=1}^n \sum_{1 \leq i_1 \leq \dots \leq i_{m-1} \leq i_m = n} 1 \\ &= f_{m,n-1} + f_{m-1,n} \end{aligned}$$

我们来考虑

$$f_{n,m} = f_{n-1,m} + f_{n,m-1} + a_{n,m}$$

这样的递推式，其中初值 $f_{\cdot,0}, f_{0,\cdot}$ 任意指定。

注意到 $a_{i,j}$ 对 $f_{n,m}$ 的贡献系数是： $g_{n,m} = g_{n-1,m} + g_{n,m-1}$ 在初值为 $g_{0,0} = 1, g_{i,0} = g_{0,i} = 0$ 时， $g_{n-i,m-j}$ 处的值。

通过转组合意义，可以发现 $g_{n,m}$ 就是 $(0,0)$ 向右向上走到 (n,m) 的方案数。所以 $a_{i,j}$ 对 $f_{n,m}$ 的贡献系数为 $\binom{n-i+m-j}{n-i}$ 。初值 $f_{i,0}$ 和 $f_{0,i}$ 的贡献系数同理。

TODO：我们在第五章再来处理这个问题

$$\min(a_1, \dots, a_n) = \sum_{i \geq 1} [\min(a_1, \dots, a_n) \geq i] = \sum_{i \geq 1} [\forall k, a_k \geq i]$$

$$\max(a_1, \dots, a_n) = \sum_{i \geq 1} [\exists k, a_k \geq i]$$

这样可以把最值问题转成判定问题来计数。

另外，把 $f(i)$ 写成 $\sum_{1 \leq k \leq i} f(k) - f(k-1)$ 常常是一个有效的思路（即使我们并没有在使用分部求和法！）。比如把 x 写成 $\sum_i [1 \leq i \leq x]$ 就是最常见的一个。

把 $(\sum_i f(i))^m$ 写成 $\sum_{i_1, i_2, \dots, i_m} f(i_1)f(i_2) \cdots f(i_m)$ 常常是有用的。

如果 X 形如“满足 xxx 条件的元素的个数”，那么把 X 写成 $\sum_x [x \text{ 满足条件}]$ 常常是有效的。这在**期望**中特别常见（称为“指示器随机变量”）。

一种容斥

某些问题要求我们计数

$$\sum (1 - [P_1])(1 - [P_2]) \cdots (1 - [P_k]).$$

那么就可以拆括号变成

$$\sum_{S \subseteq [k]} (-1)^{|S|} \sum \prod_{i \in S} [P_i],$$

其中 $\sum \prod_i [P_i]$ 往往是容易算的。

我觉得这个比那种标准的容斥要好理解和记忆得多。

在上一页中, 很多时候 $\sum \prod_{i \in S} [P_i]$ 只和 $|S|$ 有关。记这个和为 $N(|S|)$, 此时有

$$\begin{aligned} & \sum_{S \subseteq [k]} (-1)^{|S|} \sum_{i \in S} \prod [P_i] \\ &= \sum_{S \subseteq [k]} (-1)^{|S|} N(|S|) \\ &= \sum_i \binom{k}{i} (-1)^i N(i) \end{aligned}$$

二项式反演

记 $g_i = \sum_{|S|=i} \sum \prod_{i \in S} [P_i]$ 为 “钦定了 i 个性质的方案数”，那么就有

$$\begin{aligned} f_k &= \text{恰好满足 } k \text{ 个性质的方案数} = \sum_{|S|=k} \sum \prod_{i \in S} [P_i] \prod_{j \notin S} (1 - [P_j]) \\ &= \sum_{|S|=k} \sum \left(\prod_{i \in S} [P_i] \right) \sum_{T \subseteq \bar{S}} (-1)^{|T|} \prod_{j \in T} [P_j] \\ &= \sum_{|S|=k} \sum_{T' \supseteq S} (-1)^{|T'| - |S|} \sum \prod_{i \in T'} [P_i] \quad (T' = S \uplus T) \\ &= \sum_n \left(\sum_{|T'|=n} \sum \prod_{i \in T'} [P_i] \right) \left(\sum_{S \subseteq T', |S|=k} (-1)^{|T'| - |S|} \right) \\ &= \sum_n g_n \binom{n}{k} (-1)^{n-k} \end{aligned}$$

二项式反演 (续)

练习：反过来用 f 表示 g 。

设 $h_i = \sum_{|S|=i} \sum \prod_{i \notin S} [\overline{P_i}]$ 为 “至多满足 i 个性质的方案数”，找出 h 和 f 之间的关系。

分部求和法：另一种视角

$$\begin{aligned}\sum_{i=1}^n a_i b_i &= \sum_{i=1}^n a_i \left(b_0 + \sum_{j=1}^i (b_j - b_{j-1}) \right) \\&= b_0 \sum_{i=1}^n a_i + \sum_{j=1}^n (b_j - b_{j-1}) \sum_{i=j}^n a_i \\&= b_0 \sum_{i=1}^n a_i + \sum_{j=1}^n (b_j - b_{j-1}) \left(\sum_{i=1}^n a_i - \sum_{i=1}^{j-1} a_i \right) \\&= b_n \sum_{i=1}^n a_i - \sum_{j=1}^n (b_j - b_{j-1}) \sum_{i=1}^{j-1} a_i\end{aligned}$$

(注意以上的第二行和第四行都是可用的结论)

也就是说, 对 $a_i b_i$ 求和可以转为对 $(\sum a_i)(\Delta b_i)$ 求和

例子

给一棵树，点有点权，求所有路径 $\min$ 的 k 次方的和。(hint: 先考虑 $k = 1$)

$1, \dots, n$ 的所有排列的逆序对数之和是多少？

长为 n 的序列，每个都是 1 到 m 中的数。一个序列的价值为其中不同数的数量，求所有序列的价值和。(有封闭形式)

长为 n 的序列，每个都是 1 到 m 中的数。一个序列的价值为不同数的数量的 k 次方，求所有序列的价值和。(你可以认为 k 非常小或者干脆就是个常数，虽然我们实际上至少可以做到 $O(k(\log k + \log n))$)

长为 n 的序列，每个都是 1 到 m 中的数。要求 $1, \dots, k$ 必须在序列中出现，求方案数。(事实上这是上一个问题的 $O(k \log k)$ 做法的一个子问题，你只需要给出一个 $O(k^2)$ 的做法)

路径 $\min$ 的 k 次方和

$$\begin{aligned}\sum_{\text{路径 } P} \min(P)^k &= \sum_{\text{路径 } P} \sum_{i=1}^{\min(P)} i^k - (i-1)^k \\&= \sum_{i \geq 1} (i^k - (i-1)^k) \sum_{\text{路径 } P} [\min(P) \geq i] \\&= \sum_{i \geq 1} (i^k - (i-1)^k) \sum_{\text{路径 } P} [P \text{ 是只保留权值 } \geq i \text{ 的点形成的森林中的一条路径}] \\&= \sum_{i \geq 1} (i^k - (i-1)^k) \sum_B [B \text{ 是只保留权值 } \geq i \text{ 的点形成的森林中的一个连通块}] \binom{|B|+1}{2}\end{aligned}$$

假设点权从大到小为 $w_m, \dots, w_1$ 。在 $i \in [w_i, w_{i+1})$ 时第二个求和的结果都相同，而 i 从 w_i 减小到 $w_i - 1$ 时连通块会发生合并。并查集维护连通块大小即可。

路径 $\min$ 的 k 次方和：另解

也可以直接枚举 $\min$ 的取值：

$$\begin{aligned}\sum_{\text{路径 } P} \min(P)^k &= \sum_w w^k \sum_{\text{路径 } P} [\min(P) = w] \\ &= \sum_w w^k \sum_{\text{路径 } P} ([\min(P) \geq w] - [\min(P) > w])\end{aligned}$$

剩下的过程是一样的。通过分部求和法也不难发现这两个式子相等。

排列逆序对数之和

$$\begin{aligned}\sum_{\pi} \text{inv}(\pi) &= \sum_{\pi} \sum_{i < j} [\pi_i > \pi_j] \\ &= \sum_{i < j} \sum_{\pi} [\pi_i > \pi_j] \\ &= \sum_{i < j} \frac{n!}{2} \\ &= \frac{n!}{2} \binom{n}{2}\end{aligned}$$

其中第二行到第三行使用了排列的对称性，即 $\pi_i > \pi_j$ 和 $\pi_i < \pi_j$ 的排列个数是一样多的。

$$\begin{aligned}\sum_a w(a) &= \sum_a \sum_{i=1}^m [i \text{ 在 } a \text{ 中出现}] \\&= \sum_{i=1}^m \sum_a [i \text{ 在 } a \text{ 中出现}] \\&= \sum_{i=1}^m \sum_a (1 - [i \text{ 不在 } a \text{ 中出现}]) \\&= \sum_{i=1}^m (m^n - (m-1)^n) \\&= m(m^n - (m-1)^n)\end{aligned}$$

序列价值和 I: 另解

$$\begin{aligned}\sum_a w(a) &= \sum_a \sum_{i=1}^n [a_i \text{ 这个值在 } a \text{ 中首次出现}] \\&= \sum_a \sum_{i=1}^n [a_i \text{ 这个值在 } \{a_1, \dots, a_{i-1}\} \text{ 中没有出现}] \\&= \sum_{i=1}^n \sum_{x=1}^m \sum_a [a_i = x] [x \text{ 在 } \{a_1, \dots, a_{i-1}\} \text{ 中没有出现}] \\&= \sum_{i=1}^n \sum_{x=1}^m (m-1)^{i-1} m^{n-i} = \sum_{i=1}^n (m-1)^{i-1} m^{n-i+1} \\&= m^n \frac{1 - ((m-1)/m)^n}{1 - (m-1)/m} = m(m^n - (m-1)^n)\end{aligned}$$

序列价值和 II

$$\begin{aligned}\sum_a w(a)^k &= \sum_a \sum_{1 \leq i_1, i_2, \dots, i_k \leq m} [i_1, \dots, i_k \text{ 都在 } a \text{ 中出现}] \\ &= \sum_{1 \leq i_1, i_2, \dots, i_k \leq m} \sum_a [i_1, \dots, i_k \text{ 都在 } a \text{ 中出现}]\end{aligned}$$

观察小的情况，可以发现

$$\sum_a [i_1, \dots, i_k \text{ 都在 } a \text{ 中出现}]$$

只与 $i_1, \dots, i_k$ 中不同值的个数相关。并且，假设恰有 t 个不同值，那么由对称性有

$$\sum_a [i_1, \dots, i_k \text{ 都在 } a \text{ 中出现}] = \sum_a [1, \dots, t \text{ 都在 } a \text{ 中出现}]$$

枚举 $i_1, \dots, i_k$ 中不同值的个数:

$$\begin{aligned} & \sum_{1 \leq i_1, i_2, \dots, i_k \leq m} \sum_a [i_1, \dots, i_k \text{ 都在 } a \text{ 中出现}] \\ &= \sum_t \sum_{1 \leq i_1, i_2, \dots, i_k \leq m} [i_1, \dots, i_k \text{ 恰有 } t \text{ 个不同值}] \sum_a [i_1, \dots, i_k \text{ 都在 } a \text{ 中出现}] \\ &= \sum_t \left(\sum_{1 \leq i_1, i_2, \dots, i_k \leq m} [i_1, \dots, i_k \text{ 恰有 } t \text{ 个不同值}] \right) \left(\sum_a [1, \dots, t \text{ 都在 } a \text{ 中出现}] \right) \end{aligned}$$

前面这部分是

$$\sum_{1 \leq x_1 < x_2 < \dots < x_t \leq m} \sum_{1 \leq i_1, i_2, \dots, i_k \leq m} [\{i_1, \dots, i_k\} \text{ 这个集合等于 } \{x_1, \dots, x_t\}].$$

由对称性,

$$\sum_{1 \leq i_1, i_2, \dots, i_k \leq m} [\{i_1, \dots, i_k\} = \{x_1, \dots, x_t\}] = \sum_{1 \leq i_1, i_2, \dots, i_k \leq m} [\{i_1, \dots, i_k\} = \{1, \dots, t\}].$$

接下来

$$\begin{aligned} & \sum_{1 \leq x_1 < x_2 < \dots < x_t \leq m} \sum_{1 \leq i_1, i_2, \dots, i_k \leq m} [\{i_1, \dots, i_k\} \text{ 这个集合等于 } \{1, \dots, t\}] \\ &= \binom{m}{t} \sum_{1 \leq i_1, i_2, \dots, i_k \leq t} [1, \dots, t \text{ 都在 } (i_1, \dots, i_k) \text{ 中出现}]. \end{aligned}$$

注意到这和后半部分是相同形式的问题。

令

$$f(n, m, k) = \sum_{1 \leq i_1, \dots, i_n \leq m} [1, \dots, k \text{ 都在 } (i_1, \dots, i_n) \text{ 中出现}].$$

容斥:

$$\begin{aligned} f(n, m, k) &= \sum_{1 \leq i_1, \dots, i_n \leq m} \prod_{j=1}^k (1 - [j \text{ 在 } \{i_1, \dots, i_n\} \text{ 中不出现}]) \\ &= \sum_{1 \leq i_1, \dots, i_n \leq m} \sum_{S \subseteq \{1, \dots, k\}} (-1)^{|S|} [\forall x \in S, x \text{ 在 } \{i_1, \dots, i_n\} \text{ 中不出现}] \\ &= \sum_{S \subseteq \{1, \dots, k\}} (-1)^{|S|} \sum_{1 \leq i_1, \dots, i_n \leq m} [\forall x \in S, x \text{ 在 } \{i_1, \dots, i_n\} \text{ 中不出现}] \\ &= \sum_{S \subseteq \{1, \dots, k\}} (-1)^{|S|} (m - |S|)^n \\ &= \sum_t \binom{k}{t} (-1)^t (m - t)^n \end{aligned}$$

现在

$$\sum_t \left(\sum_{1 \leq i_1, i_2, \dots, i_k \leq m} [i_1, \dots, i_k \text{ 恰有 } t \text{ 个不同值}] \right) \left(\sum_a [1, \dots, t \text{ 都在 } a \text{ 中出现}] \right) \\ = \sum_{t=0}^k f(k, t, t) f(n, m, t)$$

而

$$f(n, m, k) = \sum_t \binom{k}{t} (-1)^t (m-t)^n$$

可以 $O(k)$ 计算。故本题可以 $O(k^2)$ 完成。实际上，通过卷积也可以 $O(k \log k)$ 完成。

事实上， $f(n, k, k)$ 就是第二类斯特林数 $k! \left\{ \begin{smallmatrix} n \\ k \end{smallmatrix} \right\}$ ：把 $1, \dots, k$ 视作有标号盒子， $i_1, \dots, i_n$ 视作球即可。

序列价值和 II: 另解

如果在一开始就枚举序列的价值, 就得到

$$\begin{aligned}\sum_w w^k \sum_a [w(a) = w] &= \sum_w w^k \sum_a [a \text{ 中恰有 } w \text{ 个不同值}] \\ &= \sum_w w^k \binom{m}{w} \sum_a [a \text{ 中出现的元素恰好是 } \{1, \dots, w\}] \\ &= \sum_w w^k \binom{m}{w} f(n, w, w)\end{aligned}$$

但是这个并不能直接用于计算, 因为 w 有意义的范围高达 $[0, \min(n, m)]$ 。但是我们可以用前面得到的 f 的公式继续化简。TODO: 我们在学习第五章之后再来处理这个式子。

例子：变式

给一个 0 到 $n-1$ 的排列，求所有区间的 mex 的 k 次方的和。 $\text{mex}(S)$ 为 S 中最小的没有出现过的自然数。(Bonus: 排列区间 $\rightarrow$ 序列区间; 排列区间 $\rightarrow$ 树路径 (权值为排列); 树路径 (不假设权值为排列能做吗? 或者证明 hardness))

$1, \dots, n$ 的所有排列 π 的**严格前缀 max** 数量之和是多少? 一个位置 i 是严格前缀 max 当且仅当 $\forall j < i. \pi_i > \pi_j$ 。

长为 n 的序列，每个都是 1 到 m 中的数。定义一个序列 a 的权值 $w(a)$ 为 a 中不同数的数量。完成以下两个问题中的至少一个 (需要 $O(k^2)$):

- 求所有序列的 $(w(a))^k$ 的和
- 给定 k 次多项式 f ，求所有序列的 $f(w(a))$ 的和

$$\sum_{l=1}^n \sum_{r=1}^n \sum_{i=l}^r a_i \sum_{i=l}^r b_i.$$

需要 $O(n)$

$$\sum_{i=1}^n i^k a^i$$

需要 $O(k^2)$.

分别使用扰动法和分部求和法完成

设 $S_n = \sum_{i=1}^n i^k a^i$, 那么首先

$$S_n = \sum_{i=1}^n (i+1)^k a^{i+1} - (n+1)^k a^{n+1} + 1^k a^1.$$

扰动法的关键一步在于把 $(i+1)^k a^{i+1}$ 表示成关于 i 而不是 $i+1$ 的表达式。在遇到和的幂时, 我们的第一反应就是二项式定理:

$$(i+1)^k a^{i+1} = a \sum_j \binom{k}{j} i^j a^i.$$

把这个带到扰动法中就得到

$$S_n = \sum_{i=1}^n a \sum_j \binom{k}{j} i^j a^i = a \sum_j \binom{k}{j} \sum_{i=1}^n i^j a^i.$$

(要牢记我们的基本准则 “提出无关项” 和 “交换求和号”).

现在，我们发现内部的求和和 S_n 很像，只是 k 换成了 j 。这种时候我们往往可以尝试设出辅助递归项 $S_{n,j}$ ，而 $S_{n,j}$ 关于 j 的递归具有同样的模式。

特别地，在 $a = 1$ 时无法得到 $S_{n,k}$ 的方程，而是 $S_{n,k-1}$ 的方程。然而，这并不影响我们能够递推求解 $S_{n,j}$ 的事实。

这样我们就得到了 $O(k^2)$ 的做法。而如果我们挥动线性递推重锤，我们可以得到一个 $O(k)$ 的做法。具体来说：

- 线性递推告诉我们结果具有 $a^n P(n) + C$ 的形式，其中 $P(n)$ 是一个 k 次多项式， C 是常数
- 先递推计算出 $S_0, \dots, S_k$ ，然后 $P(i) = a^{-i}(S_i - C)$
- 使用 Lagrange 插值和直接递推两种方式分别计算 $P(k+1)$ ，从而解出 C
- 使用 Lagrange 插值得到 $P(n)$ 和 S_n

分部求和法

自行尝试

普通幂和下降幂的互相转换

我们可以在 $O(k^2)$ (甚至 $O(k \log^2 k)$) 内在普通多项式和下降幂多项式之间相互转换。

做法一：拉格朗日插值

做法二：可以在 $O(k)$ 内从 x^{k-1} 计算 x^k ，从而 $O(k^2)$ 计算所有 $x^1, \dots, x^k$ 。反过来，可以类似多项式除法从高到低计算 $\sum_i a_i x^i$ 转到 $\sum_i b_i x^i$ 后的各项系数。

做法三：

$$x^k = \sum_i \left\{ \begin{matrix} k \\ i \end{matrix} \right\} x^i, \quad x^k = \sum_i \left[\begin{matrix} k \\ i \end{matrix} \right] x^i (-1)^{k-i}$$

斯特林数

我如何在不知道斯特林数的定义的情况下计算斯特林数？我们有 $x^k = \sum_i \left\{ \begin{smallmatrix} k \\ i \end{smallmatrix} \right\} x^i$ ，而另一方面有

$$\begin{aligned} x^k &= x \cdot x^{k-1} = \sum_i \left\{ \begin{smallmatrix} k-1 \\ i \end{smallmatrix} \right\} x \cdot x^i = \sum_i \left\{ \begin{smallmatrix} k-1 \\ i \end{smallmatrix} \right\} \cdot x^i (x - i + i) \\ &= \sum_i \left\{ \begin{smallmatrix} k-1 \\ i \end{smallmatrix} \right\} \cdot (ix^i + x^{i+1}) \\ &= \sum_i \left(\left\{ \begin{smallmatrix} k-1 \\ i-1 \end{smallmatrix} \right\} + i \left\{ \begin{smallmatrix} k-1 \\ i \end{smallmatrix} \right\} \right) x^i. \end{aligned}$$

比较系数就得到 $\left\{ \begin{smallmatrix} k \\ i \end{smallmatrix} \right\} = \left\{ \begin{smallmatrix} k-1 \\ i-1 \end{smallmatrix} \right\} + i \left\{ \begin{smallmatrix} k-1 \\ i \end{smallmatrix} \right\}$ ，另一个同理

二项式定理 (负指数)

计算

$$\sum_{i=0}^n i^{\overline{k}} r^i$$

假设 $0 < r < 1$, 令 $n = \infty$, 然后重新审视你的结果

三角形：直接的推法 1

$$\begin{aligned} & \sum_{i,j,k} [1 \leq i \leq j \leq k \leq n][i + j + k = n][i + j > k] \\ &= \sum_{i,j} [1 \leq i \leq j \leq n - i - j][i + j > n - i - j] \\ &= \sum_{1 \leq i \leq j} [i \leq n - 2j][i > \frac{n}{2} - j] \\ &= \sum_{j \geq 1} \sum_{i \geq 1} [\frac{n}{2} - j < i \leq \min(j, n - 2j)] \end{aligned}$$

接下来分类讨论 $\min(j, n - 2j)$ 是哪个。当 $j \leq n - 2j$ 也就是 $j \leq n/3$ 时是 j ，否则也就是 $j > n/3$ 时是 $n - 2j$ 。

$j \leq n/3$ 的部分如下:

$$\sum_{1 \leq j \leq n/3} \sum_{i \geq 1} [\frac{n}{2} - j < i \leq j].$$

在 i 的合法区间非空, 也就是 $j < n/4$ 时, 合法的 i 有 $j - \lfloor n/2 - j \rfloor = 2j - \lfloor n/2 \rfloor$ 个。所以这部分是

$$\sum_{n/4 < j \leq n/3} (2j - \lfloor n/2 \rfloor),$$

等差数列求和。

$j > n/3$ 的部分如下:

$$\sum_{n/3 < j} \sum_{i \geq 1} [\frac{n}{2} - j < i \leq n - 2j].$$

在 i 的合法区间非空, 也就是 $j < n/2$ 时, 合法的 i 有 $n - 2j - \lfloor n/2 - j \rfloor = \lceil n/2 \rceil - j$ 个。所以这部分是

$$\sum_{n/3 < j < n/2} (\lceil n/2 \rceil - j),$$

同样等差数列求和。

三角形：直接的推法 2

在最开始的条件转化中，除了先对 j 再对 i 求和外，也可能会变成先对 i 求和再对 j 求和。注意到这样子势必会引入 $j \leq \frac{n-i}{2}$ 这样的条件，从而内层对 j 的求和会带上 $\lfloor (n-i)/2 \rfloor$ 这样的东西。回顾我们的第一条化简准则——保持求和的条件简单，这样的转化不应该首先被考虑。首先考虑的应该是计算不等式中不带系数的 i 的合法区间。

不过，即使出现了 $\lfloor i/2 \rfloor$ 这样的求和也不要担心，我们可以将其写成 $\frac{i - (i \bmod 2)}{2}$ 的形式，而 $i \bmod 2 = [i \text{ 是奇数}]$ 。所以在计算 $\sum_i T(i)(i \bmod 2)$ 时，可以做代换 $i = 2k + 1$ 变成 $\sum_{2k+1} T(2k + 1)$ 。

我们将在第三章见到更多更一般的处理带有取整的求和的方法。

三角形：容斥的推法

使用容斥可以得到简单一些的推法。

$$\begin{aligned} & \sum_{i,j,k} [1 \leq i \leq j \leq k \leq n][i+j+k=n][i+j > k] \\ &= \sum_{i,j,k} [i+j+k=n][1 \leq i \leq j \leq n-i-j \leq n](1 - [i+j \leq n-i-j]) \\ &= \sum_{1 \leq i \leq j} [j \leq n-i-j] - \sum_{1 \leq i \leq j} [j \leq n-i-j][i+j \leq n-i-j] \end{aligned}$$

前半部分是

$$\begin{aligned}& \sum_{1 \leq i \leq j} [j \leq n - i - j] \\&= \sum_{1 \leq j} \sum_{i=1}^j [i \leq n - 2j] \\&= \sum_{1 \leq j \leq n/3} \sum_{i=1}^j 1 + \sum_{n/3 < j} \sum_{i=1}^{n-2j} 1 \\&= \sum_{1 \leq j \leq \lfloor n/3 \rfloor} j + \sum_{j=\lfloor n/3 \rfloor + 1}^{\lfloor n/2 \rfloor} (n - 2j)\end{aligned}$$

等差数列求和。

后半部分是

$$\begin{aligned} \sum_{1 \leq i \leq j} [j \leq n - i - j][i + j \leq n - i - j] &= \sum_{1 \leq i \leq j} [i + j \leq n - i - j] \\ &= \frac{1}{2} \left(\sum_{1 \leq i, j} [i + j \leq n - i - j] + \sum_{1 \leq i = j} [i + j \leq n - i - j] \right) \\ &= \frac{1}{2} \left(\sum_{1 \leq i, j} \sum_d [i + j = d][i + j \leq n/2] + \sum_{1 \leq i = j} [i + j \leq n/2] \right) \\ &= \frac{1}{2} \left(\sum_d \sum_{1 \leq i, j} [i + j = d][d \leq n/2] + \sum_{1 \leq i} [i \leq n/4] \right) \\ &= \frac{1}{2} \left(\sum_{d=2}^{\lfloor n/2 \rfloor} (d-1) + \lfloor n/4 \rfloor \right) \end{aligned}$$

同样是等差数列求和

三角形：对称性的推法

首先，注意到 $P(i, j, k) = [i + j + k = n][i, j, k \text{ 构成三角形}]$ 是完全对称的。如果你知道 Burnside 引理，或者尝试使用对称性并手算系数的话，可以发现

$$\sum_{1 \leq i \leq j \leq k} P(i, j, k) = \frac{1}{6} \left(2 \sum_{1 \leq i=j=k} P(i, j, k) + 3 \sum_{1 \leq i=j, k} P(i, j, k) + \sum_{1 \leq i, j, k} P(i, j, k) \right).$$

接下来，注意到

$$\begin{aligned} P(i, j, k) &= [i + j + k = n][i, j, k \text{ 构成三角形}] \\ &= [i + j + k = n][i + j + k > 2 \max(i, j, k)] \\ &= [i + j + k = n][\max(i, j, k) < \frac{n}{2}] \\ &= [i + j + k = n][i, j, k < \frac{n}{2}] \\ &= [i + j + k = n](1 - [i \geq n/2])(1 - [j \geq n/2])(1 - [k \geq n/2]) \end{aligned}$$

注意在 $1 \leq i, j, k$ 的限制下, 拆括号后如果一项中出现了两个或更多 $[i \geq n/2]$ 的乘积, 那么这个项不会有贡献 (因为此时 $i + j + k$ 必定超过 n)。类似地, $i = j, k$ 这个求和里面不会拆出来 $[i \geq n/2]$ 的项。所以我们有

$$\sum_{1 \leq i=j=k} P(i, j, k) = \sum_{1 \leq i} [3i = n] = [3 \mid n],$$

$$\begin{aligned} \sum_{1 \leq i=j,k} P(i, j, k) &= \sum_{1 \leq i=j,k} [i + j + k = n](1 - [k \geq n/2]) \\ &= \sum_{1 \leq i} [n - 2i \geq 1] - \sum_{1 \leq i} [n/2 \leq n - 2i] \\ &= \left\lfloor \frac{n-1}{2} \right\rfloor - \left\lfloor \frac{n}{4} \right\rfloor. \end{aligned}$$

以及

$$\begin{aligned}& \sum_{1 \leq i, j, k} P(i, j, k) \\&= \sum_{1 \leq i, j, k} [i + j + k = n](1 - [i \geq n/2] - [j \geq n/2] - [k \geq n/2]) \\&= \sum_{1 \leq i, j, k} [i + j + k = n] - 3 \sum_{1 \leq i, j, k} [i + j + k = n][i \geq n/2] \\&= \sum_{1 \leq i, j, k} [i + j + k = n] - 3 \sum_{1 \leq (i - \lceil n/2 \rceil + 1), j, k} [(i - \lceil n/2 \rceil + 1) + j + k = n - (\lceil n/2 \rceil - 1)] \\&= \binom{n-1}{2} - 3 \binom{\lfloor n/2 \rfloor - 2}{2},\end{aligned}$$

这个做法可以比较方便地从三角形扩展到一般的 k 边形（在旋转和翻转下数等价类）

- 底和顶 (上/下取整): 定义、等价不等式
- 含有取整的求和: 变量代换
- mod 的部分应用

第三章习题

需要写下来的：1 3 7 9 12 16 20 22 31 36 39

补充练习：洛谷 P2260 P6826 P5170 P5591

同余等差数列的结构

序列 $(ik \bmod n)_{i=0}^{n-1}$ 长什么样子?

当 $\gcd(k, n) = 1$ 时, 这是 $0, \dots, n-1$ 的一个排列。这是因为对任何 $0 \leq i < j < n$, $n \nmid (j-i)k$, 从而 $ik \bmod n \neq jk \bmod n$ 。

当 $\gcd(k, n) = d$ 时, $ik \bmod n = d(i(k/d) \bmod (n/d))$, 而 $\gcd(k/d, n/d) = 1$ 。所以此时序列是 $0, \dots, n/d-1$ 的一个排列重复 d 次。

处理取整的一些技巧

技巧一: $\left\lfloor \frac{n}{m} \right\rfloor = \frac{n}{m} - \frac{n \bmod m}{m}$ 。另外, 在分子比较复杂时, 提前分离出整除的部分常常是方便的, 如

$$\begin{aligned}\sum_{i=0}^{m-1} \left\lfloor \frac{n+i}{m} \right\rfloor &= \sum_{i=0}^{m-1} \left\lfloor \frac{m \lfloor n/m \rfloor + (n \bmod m) + i}{m} \right\rfloor \\&= \sum_{i=0}^{m-1} \left\lfloor \frac{n}{m} \right\rfloor + \left\lfloor \frac{(n \bmod m) + i}{m} \right\rfloor \\&= m \left\lfloor \frac{n}{m} \right\rfloor + \sum_{i=0}^{m-1} [(n \bmod m) + i \geq m] \\&= m \left\lfloor \frac{n}{m} \right\rfloor + (n \bmod m) \\&= n\end{aligned}$$

技巧二：在处理含有取整的求和时，可以引入一个新的求和变量替代取整，然后将原本的取整变为一个不含取整的条件括号。形式化地说，就是

$$\lfloor x \rfloor = \sum_k k[\lfloor x \rfloor = k] = \sum_k k[k \leq x < k + 1].$$

技巧三：把一个量写成差分的前缀和再次发挥作用。例如

$$\lfloor x \rfloor^2 = \sum_{i \geq 1} (i^2 - (i-1)^2)[i \leq \lfloor x \rfloor] = \sum_{i \geq 1} (i^2 - (i-1)^2)[i \leq x].$$

(在使用技巧二和三时，要记得交换求和总是很有用)

技巧四：另一种差分技巧。一会会展示。

$\lfloor n/i \rfloor$ 只有 $\leq 2\sqrt{n}$ 种不同的取值 (要么 $n/i \leq \sqrt{n}$, 要么 $i \leq \sqrt{n}$)

如何得知一个等值段的端点? 假设目前在 i , 我们希望知道最大的满足 $\lfloor n/j \rfloor = \lfloor n/i \rfloor$ 的 j 。记 $t = \lfloor n/i \rfloor$, 则 $\lfloor n/j \rfloor \geq t \iff n/j \geq t \implies j \leq n/t \implies j \leq \lfloor n/t \rfloor$ 。所以最大的满足 $\lfloor n/i \rfloor = \lfloor n/j \rfloor$ 的 j 即为 $\lfloor n/t \rfloor$ 。

在同一个 $\lfloor n/i \rfloor$ 的等值区间内, 可以做一些批量的操作 (如区间加/求和等)

$$\sum_{i=1}^n f(\lfloor n/i \rfloor) g(i).$$

根据 $\lfloor n/i \rfloor$ 的值划分等值段，每段内的求和都形如 $\sum_{i=L}^R f(t)g(i) = f(t) \sum_{i=L}^R g(i)$ ，那么整个式子可以在 $O(\sqrt{n} \cdot \text{time}(\text{计算一次 } g \text{ 的区间和}))$ 内完成。 g 的区间和通常可以快速计算（比如提前预处理前缀和）。

典型的无需预处理也可以快速计算的 g ： $P(i)r^i$ （其中 P 是低次多项式）

不妨设 $n \leq m$ 。首先容斥掉 $i \neq j$ 的限制得到

$$\left(\sum_{i=1}^n n \bmod i \right) \left(\sum_{j=1}^m m \bmod j \right) - \sum_{i=1}^n (n \bmod i)(m \bmod i).$$

先看前半部分。乘积的两部分是同一种形式，考虑如何计算其中一个。把 $n \bmod i$ 写作 $n - i \lfloor n/i \rfloor$ ，则每个 $\lfloor n/i \rfloor$ 的等值段内的 $\sum_i n - i \lfloor n/i \rfloor$ 是容易计算的。

再看后半部分，类似地改写 mod 得到

$$\sum_{i=1}^n (n - i \lfloor n/i \rfloor)(m - i \lfloor m/i \rfloor).$$

扩展前面等值段的想法，考虑 i 的段划分，满足在每一段上 $\lfloor n/i \rfloor$ 相同、 $\lfloor m/i \rfloor$ 也相同。注意这些段的断点是由「 $\lfloor n/i \rfloor$ 的段划分断点 $\cup$ $\lfloor m/i \rfloor$ 的段划分断点」形成的，所以总共的段数不超过 $2(\sqrt{n} + \sqrt{m})$ 。

实现时，只要每次在 $\lfloor n/i \rfloor$ 的下一个变值点和 $\lfloor m/i \rfloor$ 的下一个变值点中取一个较小的跳过去即可。

另一个例子

求

$$\sum_{i \geq 1} \left\lfloor \frac{n}{i^2} \right\rfloor.$$

$$n \leq 10^{18}$$

注意到要么 $i \leq \sqrt[3]{n}$, 要不然 $i > \sqrt[3]{n} \Rightarrow n/i^2 < \sqrt[3]{n}$, 所以 $\lfloor n/i^2 \rfloor$ 只有不超过 $2\sqrt[3]{n}$ 个不同的值。

下一个变值点可以这样算: $\lfloor n/i^2 \rfloor \geq t \iff n/i^2 \geq t \implies i \leq \sqrt{n/t} \implies i \leq \lfloor \sqrt{n/t} \rfloor$. 所以最大的满足 $\lfloor n/i^2 \rfloor = \lfloor n/j^2 \rfloor$ 的 j 即为 $\lfloor \sqrt{n/t} \rfloor$ 。

$$\lfloor \frac{n}{i} \rfloor - \lfloor \frac{n-1}{i} \rfloor = [i|n].$$

$$S(n) = S(0) + \sum_{i=1}^n (S(i) - S(i-1)).$$

$$\begin{aligned}\sum_{i \geq 1} \left\lfloor \frac{N}{i} \right\rfloor &= \sum_{n=1}^N \sum_{i \geq 1} \left\lfloor \frac{n}{i} \right\rfloor - \left\lfloor \frac{n-1}{i} \right\rfloor \\ &= \sum_{n=1}^N \sum_{i \geq 1} [i|n] \\ &= \sum_{n=1}^N d(n),\end{aligned}$$

其中 $d(n)$ 为 n 的约数个数。

对所有 $1 \leq n \leq N$ 求

$$S(n) = \sum_{i=1}^n a_i \left\lfloor \frac{n}{i} \right\rfloor.$$

$$N \leq 10^6$$

$$\begin{aligned}
\nabla S(n) &= S(n) - S(n-1) \\
&= \sum_{i \geq 1} a_i \left(\left\lfloor \frac{n}{i} \right\rfloor - \left\lfloor \frac{n-1}{i} \right\rfloor \right) \\
&= \sum_{i \geq 1} a_i [i|n] \\
&= \sum_{i|n} a_i.
\end{aligned}$$

枚举 i , 对所有 i 的倍数 n 令 $\nabla S(n) \leftarrow \nabla S(n) + a_i$ 。最后再做一次前缀和还原出 S 。复杂度 $O(nH_n) = O(n \ln n)$ 。

另外, $\nabla S(n) = A * \mathbf{1}$ 实际上是狄利克雷前缀和, 可以 $O(n \log \log n)$ 计算。

枚举 i , 则 a_i 在一个 $\lfloor n/i \rfloor$ 的等值段上都会对 $S(n)$ 造成相同的贡献 $a_i \lfloor n/i \rfloor$ 。等值段为所有的 $[ki, (k+1)i)$, 有 $O(n/i)$ 段。所以问题变成 $O(nH_n)$ 个区间加, 最后求所有值, 差分即可。

例

求

$$\sum_{i,j \geq 1} \left\lfloor \frac{n}{ij} \right\rfloor.$$

$$n \leq 10^{10}$$

另外，你能证明这是一个积性函数前缀和吗？

$$\sum_{i,j \geq 1} \left\lfloor \frac{n}{ij} \right\rfloor = \sum_{i \geq 1} \sum_{j \geq 1} \left\lfloor \frac{\lfloor n/i \rfloor}{j} \right\rfloor.$$

记 $f(n) = \sum_{j \geq 1} \lfloor n/j \rfloor$, 则原式即 $\sum_{i \geq 1} f(\lfloor n/i \rfloor)$.

枚举所有 $\lfloor n/i \rfloor$ 的等值段, 每次花费 $O(\sqrt{n/i})$ 的复杂度。总复杂度为

$$O\left(\sum_{i=1}^{\sqrt{n}} \sqrt{i} + \sum_{i=1}^{\sqrt{n}} \sqrt{n/i}\right).$$

结论

使用一些微积分知识可以得到

$$\sum_{i=1}^n i^{\alpha} \sim \begin{cases} \frac{n^{\alpha+1}}{\alpha+1} & \alpha > -1 \\ \ln n & \alpha = -1 \\ \zeta(-\alpha) = O(1) & \alpha < -1 \end{cases}.$$

另外在 $\alpha < -1$ 的时候还有

$$\sum_{i \geq n} i^{\alpha} \sim \frac{n^{\alpha+1}}{\alpha+1}.$$

所以上一页中的复杂度是

$$O\left(\sum_{i=1}^{\sqrt{n}} i^{1/2} + \sqrt{n} \sum_{i=1}^{\sqrt{n}} i^{-1/2}\right) = O((\sqrt{n})^{3/2} + \sqrt{n} \cdot (\sqrt{n})^{1/2}) = O(n^{3/4}).$$

在先前是例子中，我们已经证明了 $f(n)$ 是约数个数函数 $d(n)$ 的前缀和。对于任何积性函数，我们可以用线性筛筛出其前 n 项，进而求出其前缀和。所以，我们可以用 $O(B)$ 的复杂度预处理所有 $k \leq B$ 的 $f(k)$ 。对于 $k > B$ 的部分，我们还是用整除分块计算。假设 $B > \sqrt{n}$ 。现在，只有 $\lfloor n/i \rfloor \geq B$ 也就是 $i \leq n/B$ 的部分需要使用整除分块，总复杂度为

$$O\left(B + \sum_{i=1}^{n/B} \sqrt{n} i^{-1/2}\right) = O\left(B + n/B^{1/2}\right).$$

为寻求其最小值，我们可以令 $B = n/B^{1/2}$ ，这样得到 $B = n^{2/3}$ 。此时，上面的复杂度变成 $O(n^{2/3})$ 。

不难用差分技巧发现原式即为

$$\sum_{i=1}^N \sum_{j|i} \sum_{k|\frac{i}{j}} 1 = \sum_{i=1}^N (1 * 1 * 1)(i).$$

这是一个积性函数前缀和，可以用 zky 筛做到 $\tilde{O}^1(\sqrt{n})$ 。然而没什么实用价值。

${}^1\tilde{O}(f(n))$ 代表 $O(f(n) \log^{O(1)} f(n))$ ，用在不关心 $\log$ 因子的时候。然而，OI 中 $\log$ 因子几乎总是重要的。

单位根反演²

假设存在某个 ω_n 使得

$$\frac{1}{n} \sum_{j=0}^{n-1} (\omega_n^i)^j = [i \bmod n = 0],$$

则把 $[i \bmod n = 0]$ 用这个和式替代往往是有用的。(那么 $[i \bmod n = j]$ 呢?)

本原单位根就满足这个性质。定义如下:

定义 (单位根 (root of unity))

任何满足 $\omega_n^n = 1$ 的值 $\omega_n \in \mathbb{F}$ 都是域 $\mathbb{F}$ 中的一个 n 次**单位根**。如果额外满足 $\forall 0 < i < n, \omega_n^i \neq 1$, 那么称为 n 次**本原单位根**。

下面我们说单位根的时候都指的是本原单位根。小练习: 对于本原单位根 ω_n , 证明 $\omega_n, \omega_n^1, \omega_n^2, \dots, \omega_n^{n-1}, 1$ 为方程 $x^n = 1$ 的 n 个**不同的根**。

²这个名字应该是 OI 圈生造的, 反演在哪?

目前，你可以认为域是一个可以做加减乘除的集合。常见的域包括有理数 $\mathbb{Q}$ ，实数 $\mathbb{R}$ ，复数 $\mathbb{C}$ ，以及 OI 中最常见的有限域 $\mathbb{F}_p$ （可以认为是 $\text{mod } p$ 下做运算）。需要注意前面几个和最后一个的性质很不相同。

证明

考虑

$$\sum_{j=0}^{n-1} (\omega_n^i)^j$$

是什么。首先有 $\omega_n^i = \omega_n^{i \bmod n}$ 。所以不妨设 $0 \leq i < n$ 。当 $i = 0$ 时, 每一项都是 $\omega_n^{0 \cdot j} = 1$; 当 $i \neq 0$ 时, 这是等比数列求和

$$\frac{1 - (\omega_n^i)^n}{1 - \omega_n^i} = 0.$$

所以

$$\sum_{j=0}^{n-1} (\omega_n^i)^j = \begin{cases} n & i \bmod n = 0 \\ 0 & \text{otherwise} \end{cases} = n[i \bmod n = 0]$$

单位根的例子

± 1 是 $x^2 = 1$ 的两个根, 所以 -1 可以作为 ω_2 (但是在 mod 2 下不行)。我们有 (也许是最常见的应用)

$$[n \bmod 2 = 0] = \frac{1^n + (-1)^n}{2}, \quad [n \bmod 2 = 1] = \frac{1^n - (-1)^n}{2}.$$

在复数域 $\mathbb{C}$ 上, $e^{2\pi i/n}$ 是一个 n 次单位根。

在数论中有原根的概念。我们只考虑模素数 p 的情况: 如果 $g, g^2, \dots, g^{\varphi(n)-1}, g^{p-1} = 1$ 在 mod p 下互不相同, 则称 g 是 mod p 下的一个原根。换句话说, g 是 mod p 下的一个 $p-1$ 次单位根。例如, 3 是 998244353 的一个原根, 所以是 mod 998244353 时的 998244352 = $2^{23} \times 7 \times 17$ 次单位根。

ω_n^d 是一个 $n/\gcd(n, d)$ 次单位根。所以, 在 mod p 时, 如果 $n \mid (p-1)$, 那么 $\omega_{p-1}^{(p-1)/n} = g^{(p-1)/n}$ 就是一个 n 次单位根, 其中 g 是 mod p 下的原根。

求 $\sum_i \binom{n}{2i} x^{2i}$ 的封闭形式.

2 换成 3 或者 4 呢?

$$\begin{aligned}
\sum_i \binom{n}{2i} x^{2i} &= \sum_i [i \bmod 2 = 0] \binom{n}{i} x^i \\
&= \sum_i \binom{n}{i} x^i \cdot \frac{1^i + (-1)^i}{2} \\
&= \frac{(1+x)^n + (1-x)^n}{2}.
\end{aligned}$$

更一般地, 不难得到

$$\sum_i \binom{n}{ki+r} x^{ki+r}$$

的做法 ($0 \leq r < k$, 假设 ω_k 存在)。

与傅里叶变换的联系

离散傅里叶变换 DFT_n 事实上是在 $\text{mod}(x^n - 1)$ 下做的。换句话说,

$$\text{IDFT}_n(\text{DFT}_n(F(x))) = F(x) \bmod (x^n - 1).$$

而 $[i \bmod n = 0]$ 刚好就是 $[x^0](x^i \bmod (x^n - 1))$, 做一下上面的正逆变换刚好就是单位根反演的形式。

有时候单位根不存在，此时怎么办呢？我们以 ω_3, ω_4 为例。一般情况涉及到比较深的背景知识。

先看 ω_3 。我们把原本的数扩充成 $a + b\omega_3$ 的形式（用标准的术语来说，我们在 $\mathbb{F}[\omega_3]$ 中做运算），然后考虑加法和乘法的形式。加法就是对应项相加，而乘法是

$$(a + b\omega_3)(c + d\omega_3) = ac + (ad + bc)\omega_3 + bd\omega_3^2.$$

对于这里新出现的 ω_3^2 ，我们注意到 ω_3 满足 $1 + \omega_3 + \omega_3^2 = 0$ ，所以可以把 ω_3^2 用 $-1 - \omega_3$ 替换。

ω_4 也是类似的，只需要注意到 $\omega_4^4 - 1 = (\omega_4 - 1)(\omega_4 + 1)(\omega_4^2 + 1) = 0$ 导致 $\omega_4^2 + 1 = 0$ 。

在一般情况下, ω_k 的幂之间的关系由其**最小多项式** $M(x)$ 决定。假设其次数 $\deg M(x) = m$, 那么 $1, \omega_k, \dots, \omega_k^{m-1}$ 之间线性无关, 元素之间的乘法可以视为在 $\text{mod } M(x)$ 下做运算。

这部分可以参考域论以及 Galois 理论。

Kummer 定理

对于素数 p , 记 $v_p(n)$ 为 n 的质因子分解中 p 的指数。

定理 (Kummer)

$v_p\left(\binom{a+b}{a}\right)$ 等于 a 和 b 在 p 进制下做加法时的进位次数。

证明.

- $v_p(n!)$ 是多少?
- $v_p\left(\binom{a+b}{a}\right)$ 是多少
- $\lfloor a/p^k \rfloor + \lfloor b/p^k \rfloor$ 和 $\lfloor (a+b)/p^k \rfloor$ 的关系是什么?



$$\begin{aligned}
\sum_{i=l}^r (i-1) \sum_{j=1}^n \lceil \log_i j \rceil &= \sum_{i=l}^r (i-1) \sum_{j=1}^n \sum_{k \geq 1} [k \leq \lceil \log_i j \rceil] \\
&= \sum_{i=l}^r (i-1) \sum_{j=1}^n \sum_{k \geq 1} [k < \log_i j + 1] \\
&= \sum_{i=l}^r (i-1) \sum_{j=1}^n \sum_{k \geq 1} [i^{k-1} < j] \\
&= \sum_{k \geq 1} \sum_{i=l}^r (i-1) \max(n - i^{k-1}, 0)
\end{aligned}$$

注意到当 $k-1 > \log_2 n$ 时, 求和项一定为 0。所以 k 只需要枚举到大概 $\log_2 n$ 左右。对每个 k , 考虑 $n - i^{k-1} \geq 0$ 可以得到 i 有意义的范围, 然后里面的东西就是一些 $\sum i^k$ 的形式。

我们在前面已经讲过如何对一个固定的 n 在 $O(k^2)$ 时间内计算

$$P_k(n) = \sum_{i=1}^n i^k,$$

而现在的问题是对一些不同的 k 和 n_k 计算这个式子。事实上，我们可以在以前的推导过程中将 n 视为**变量**，这样我们就能得到关于 n 的**多项式** $P_k(n)$ 。这样，我们就可以用 $O(k)$ 的代价查询一个 n_k 的结果。

我们讲过的两种推导都需要 $O(k^3)$ 的预处理时间。而通过生成函数，我们可以得到一个 $O(k^2)$ 的预处理算法。

考虑 EGF

$$\sum_k \frac{x^k}{k!} \sum_{i=0}^{n-1} i^k = \sum_{i=0}^{n-1} e^{ix} = \frac{e^{nx} - 1}{e^x - 1}.$$

我们提前预处理伯努利数 $B(x) = \frac{x}{e^x - 1}$ 的系数。那么提取 $[x^k]$ 就得到

$$\begin{aligned} [x^k] \frac{e^{nx} - 1}{e^x - 1} &= [x^k] \left(\frac{e^{nx} - 1}{x} \right) \left(\frac{x}{e^x - 1} \right) \\ &= \sum_{i=0}^k B_{k-i} [x^i] \left(\frac{e^{nx} - 1}{x} \right) \\ &= \sum_{i=0}^k B_{k-i} \frac{n^{i+1}}{(i+1)!}. \end{aligned}$$

类欧几里得算法

- 二项式系数及相关恒等式
- 练习，练习，练习！
- 差分和二项式反演
- 生成函数初步

接下来两周内，阅读 5.1-5.4。第一周至少需要读完 5.1-5.2 并把 5.2 中所有练习自己推一遍。

习题中需要写下来的（需要对每道题自行判断目前学完的东西能不能让你完成这道题）：

2 3 4 5 8 14 15 16 35 36 37 40 41 43 45 46 47 59 61 63 64 67 70 71 73 74 76 77 79

- 加法原理: $|A \uplus B| = |A| + |B|$
- 乘法原理: $|A \times B| = |A||B|$
 - 扩展的乘法原理: 假设 S 是 $A \times V$ 的一个子集。如果对每个 $x \in A$, 都有 $\#\{y : (x, y) \in S\} = m$, 则 $|S| = m|A|$
- 减法 (容斥) 原理: $|A - B| = |A| - |A \cap B|$
- 除法原理: 假设 S 是 $A \times V$ 的一个子集。如果对每个 $x \in A$, $\#\{y : (x, y) \in S\}$ 都相等, 则 $\#\{y : (x, y) \in S\} = |S|/|A|$

洛谷 P5824 十二重计数法（由于本题实际需要 FFT，我们只列式子）

I 和 II

逐球考虑，容易用乘法原理计数。得到 $f_I(n, m) = m^n$ 和 $f_{II} = m^n$ 。

如果逐盒子考虑呢？对于 I，枚举每个盒子里的球数得到

$$\begin{aligned} & \sum_{i_1+i_2+\cdots+i_m=n} \binom{n}{i_1} \binom{n-i_1}{i_2} \binom{n-i_1-i_2}{i_3} \cdots \binom{n-i_1-\cdots-i_{m-1}}{i_m} \\ &= \sum_{i_1+i_2+\cdots+i_m=n} \frac{n!}{i_1!i_2!\cdots i_m!} \\ &= (1+1+\cdots+1)^n = m^n. \end{aligned}$$

对于 II，类似计算得到

$$\sum_{i_1+i_2+\cdots+i_m=n; i_1, i_2, \dots, i_m \leq 1} n! = \binom{m}{n} n! = m^n.$$

逐球考虑难以处理限制，不过我们可以先把限制容斥掉。容斥后，如果强制了 i 个空盒子，那么就变成 $f_I(n, m - i)$ 。所以

$$f_{III}(n, m) = \sum_i \binom{m}{i} (-1)^i f_I(n, m - i) = \sum_i \binom{m}{i} (-1)^i (m - i)^n.$$

也可以逐盒考虑，这样就是

$$\sum_{i_1 + i_2 + \cdots + i_m = n; i_1, \dots, i_m \geq 1} \frac{n!}{i_1! i_2! \cdots i_m!}.$$

仍然需要容斥掉 $i_j \geq 1$ 来计算。

考虑关于 n 的 EGF。在这三种情况下，一个盒子的 EGF 分别为 e^x 、 $1 + x$ 和 $e^x - 1$ ，所以整个 EGF 分别是 e^{mx} 、 $(1 + x)^m$ 和 $(e^x - 1)^m$ 。

标号 (一)

有区别/可区分/有标号/互不相同都指向相同的概念。一般来说，有标号是最通用和方便的说法。

球与盒子均互不相同的情况具有完整的标号，可以逐球考虑/逐盒子考虑来计数。

在球互不相同、盒子相同时，盒子的任意排列被视为同一种方案（等价类）。此时，常见的处理方法是**强制**一个顺序。

- 注意到当盒子均非空时，每个等价类的大小都为 $m!$ （也就是盒子的任意排列都对应一种不同的方案）。所以 $f_{VI}(n, m) = \frac{1}{m!} f_{III}(n, m)$ 。
- 有空盒子的情况则更复杂一些，因为空盒子的不同排列即使在盒子有标号时对应的也是同一种方案。假设有 $m - k$ 个空盒子，那么等价类大小是 $\frac{1}{\binom{m}{m-k} k!} = \frac{1}{m^{\underline{k}}}$ 。

IV, V 和 VI

由于盒子无区别，我们可以强制一个顺序。例如，按照每个盒子里装的球的最小标号升序排序（空盒子视为 0）。那么对于 IV，我们可以想象前面有一些空盒子，然后后面是一个 VI。所以

$$f_{IV}(n, m) = \sum_{i=0}^m f_{VI}(n, i).$$

对于 V，唯一的一种排法就是 $m - n$ 个空盒子后面按顺序排着装 $1, 2, \dots, n$ 的盒子。所以 $f_V(n, m) = [n \leq m]$ 。或者通过上一页中对等价类的分析，我们也可以发现 $f_V(n, m) = \frac{1}{m^n} f_{II}(n, m)$ 。

对于 VI，我们在上一页中已经得到 $f_{VI} = \frac{1}{m!} f_{III}$ 。事实上，这就是第二类斯特林数 $\left\{ \begin{smallmatrix} n \\ m \end{smallmatrix} \right\}$ 的一种定义。代入 III 的结果就得到

$$\left\{ \begin{smallmatrix} n \\ m \end{smallmatrix} \right\} = \frac{1}{m!} \sum_i \binom{m}{i} (-1)^i (m - i)^n.$$

标号 (二)

在球相同、盒子互不相同，有标号球退化为无标号的“单位”，也就是只关心每个盒子里装球的**个数**。

- 此时的一个等价类大小为 $\frac{n!}{n_1! \cdots n_m!}$ ，其中 n_i 为第 i 个盒子里的球数。
- 因为不同等价类大小不同，除法原理无法在此应用。

在球相同、盒子也相同时，盒子的任意排列被视为同一种方案。此时可以强制按照盒子中装的球数来排序。

- 此时的一个等价类在前一个问题中具有大小 $\frac{m!}{m_0! m_1! m_2! \cdots m_n!}$ ，其中 m_i 表示装了 i 个球的盒子个数。
- 同样因为不同等价类大小不同，除法原理无法在此应用。

VII, VIII 和 IX

我们可以通过如下操作建立 VII 与 IX 的双射：对于每个合法的 VII 方案，在每个盒子里放一个球即可得到一个合法的 IX 方案，并且这是一个单射；反之，对于每个合法的 IX 方案，在每个盒子里拿走一个球即可得到一个合法的 VII 方案，而这同样是一个单射。所以我们得到 $f_{\text{VII}}(n, m) = f_{\text{IX}}(n + m, m)$ 。

对于 IX，我们可以使用称为“隔板法”的双射：在 n 个球之间插 $m - 1$ 个隔板，这与 IX 的方案有一一对应。而前者的方案数显然是 $\binom{n-1}{m-1}$ ，所以 $f_{\text{IX}}(n, m) = \binom{n-1}{m-1}$ 。这样我们也得到 $f_{\text{VII}}(n, m) = \binom{n+m-1}{m-1}$ 。（也可以枚举第一个盒子放了几个球，然后使用我们以前学过的方法处理递推式）

对于 VIII，我们可以考虑哪些盒子装了球，这有 $\binom{m}{n}$ 种方案。或者，我们也可以使用上一页中的等价类大小分析来得到 $f_{\text{VIII}}(n, m) = f_{\text{II}}(n, m)/n!$ 。

在这三种情况下，一个盒子的 OGF 分别为 $\frac{1}{1-x}$ 、 $1+x$ 和 $\frac{x}{1-x}$ 。所以总的 OGF 分别为 $\frac{1}{(1-x)^m}$ 、 $(1+x)^m$ 和 $\frac{x^m}{(1-x)^m}$ 。

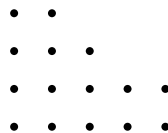
不难发现生成函数处理盒子有区别的情况是方便的。

X, XI 和 XII

对于 XI, 我们可以使用和 V 同样的分析, 或者也可以使用前面的等价类大小分析来得到同样的 $f_{\text{XI}} = [n \leq m]$ 。

对于 X 和 XII, 我们可以类比 VII 与 IX 来得到 $f_{\text{X}}(n, m) = f_{\text{XII}}(n + m, m)$ 。

对于 X, 这是一个拆分数类型的问题。此类问题的直观理解如下: 把每个球画成一个点, 把一个盒子里装的球画成一行的点, 不同盒子画在不同行并对齐, 强制盒子里装的球数从上往下不降。这样得到的点阵被称为 **Ferrers** 图。例如, 右图就是 2, 3, 5, 5 的 Ferrers 图。



从这个图中我们可以注意到其行和列是**对称**的, 而列的组合意义是盒子的容量限制。所以, 使用不超过 m 个盒子、容量无限制的方案数等于盒子数量无限制、每个盒子容量不超过 m 的方案数。

下面省略 f_X 的下标 X , 即写作 $f(n, m)$ 。首先, 我们可以通过枚举最左边有几列高度为 m , 得到

$$f(n, m) = \sum_{i \geq 0} f(n - im, m - 1).$$

通过相减抵消求和, 可以得到 $f(n, m) = f(n - m, m) + f(n, m - 1)$, 这是一个 $O(nm)$ 的 dp。或者, 我们也可以枚举最左边一列的高度, 再通过相减抵消求和得到相同的 dp。

$$f(n, m) = \sum_{i \leq m} f(n - i, i) = f(n, m - 1) + f(n - m, m)$$

这个 dp 还有另一个理解: 考虑行数够不够 m , 如果不够那么是 $f(n, m - 1)$, 如果够那么可以把第一列删掉, 这样是 $f(n - m, m)$ 。总之, 如果 $m \leq \sqrt{n}$, 我们就得到了一个 $O(n\sqrt{n})$ 的做法。

如果 $m > \sqrt{n}$, 我们可以想象 Ferrers 图的结构: 左边有若干高度 $> \sqrt{n}$ 的列 (不超过 $\sqrt{n}$ 个), 右边是高度不超过 $\sqrt{n}$ 的列。记 $B = \lfloor \sqrt{n} \rfloor + 1$ 为左边的列高度的可能最小值, 后面的讨论中都假设 B 是独立于 n 的变量。

如果我们可以 $O(n^2/B)$ 的时间内计算出所有 $g_B(i)$, 表示每列高度都在 $[B, m]$ 内的 i 个点的 Ferrers 图的数量, 那么就可以枚举左右的点数计算答案:

$$f(n, m) = \sum_i g_B(n - i) f(i, B - 1).$$

而 $f(i, B - 1)$ 可以通过上一页的 dp 在 $O(nB) = O(n\sqrt{n})$ 时间内计算。

现在来考虑 g_B 的计算。设 $g(n, k)$ 表示列数恰好为 k 且每列高度都在 $[B, m]$ 内的 Ferrers 图数量, 则 $k \leq n/B$ 。我们考虑有没有高度为 B 的列。如果有, 那么这样的方案数为 $g(n - B, k - 1)$; 如果没有 (也就是都 $> B$), 那么我们可以把最后一行删去, 所以方案数是列数恰好为 k 且每列高度都在 $[B, m - 1]$ 内的 $n - k$ 个点的 Ferrers 图数量。而这个可以用高度都在 $[B, m]$ 内的方案数减去最高高度恰好为 m 的方案数来计算, 其中前者就是 $g(n - k, k)$, 而后者可以通过删掉第一列得到 $g(n - k - m, k - 1)$ 。所以

$$g(n, k) = g(n - B, k - 1) + g(n - k, k) - g(n - k - m, k - 1).$$

再对 k 做一下前缀和就可以计算出 $g_B(n)$ 。

对全部的 n 计算

如果直接卷积, 那么需要 $O(n^2)$ 时间。然而, 如果把 $f(i, B-1)$ 作为初值来做 g 的 dp, 那么就可以在 $O(n\sqrt{n})$ 时间内算出全部的 $f(n, m)$ 。或者说, 现在 $g(n, k)$ 的含义变为:
额外列数恰好为 k 的 Ferrers 图数量, 其中额外列指的是这些高度在 $[B, m]$ 内的列。转移和之前是相同的, 只是初值变成了 $f(i, B-1)$ 。

另一种计算 $f(n, m)$ 的方法是考虑包含左下角的极大正方形。假设这个正方形的边长是 i ，那么这个正方形上面的部分就是一个高度不超过 $m - i$ 、宽度不超过 i 的 Ferrers 图，右边的部分是一个高度不超过 i 的 Ferrers 图，而正方形本身提供了 i^2 个点，所以 $i \leq \sqrt{n}$ 。上面和右边的部分都可以类似前面的 dp 计算。

可能也可以用上一页的手法算出所有 n 的结果。

考虑高为 i 的行有几个，不难写出生成函数

$$\prod_{i=1}^m \frac{1}{1-x^i}.$$

可以通过 $\ln - \exp$ 做到 $O(n \log n)$ ，或者使用 q -analog 理论得到若干种不需要 FFT 的 $O(n\sqrt{n})$ 做法。

对于整数 $x_1, \dots, x_m \geq 0$, 求

- $x_1 + x_2 + \dots + x_m = n$ 的方案数
- 在上一条的基础上, 加上 $l_i \leq x_i < r_i$ 的限制
- $x_1 \leq x_2 \leq \dots \leq x_{m-1} \leq x_m = n$ 的方案数
- 在上一条的基础上, 加上 $l_i \leq x_i - x_{i-1} < r_i$ 的限制 (设 $x_0 = 0$)

第一个问题和之前的 VII 是相同的。

通过换元 $y_i = x_i - x_{i-1}$, 可以发现第三 (四) 个问题和第一 (二) 个问题是相同的。(事实上, 我们以前已经通过和式处理的手法计算过第三个问题)

对于第二个问题, 先考虑只有下界 $x_i \geq l_i$ 的情况。通过换元 $y_i = x_i - l_i$, 我们可以将其变成第一个问题 (n 变成 $n - \sum_i l_i$)。

现在, 我们可以不失一般性地假设 $l_i = 0$ 。然后我们可以通过容斥把上界 r_i 变成下界。

$$\begin{aligned}
& \sum_{x_1+x_2+\cdots+x_m=n} \prod_i [x_i < r_i] \\
&= \sum_{x_1+x_2+\cdots+x_m=n} \prod_i (1 - [x_i \geq r_i]) \\
&= \sum_{x_1+x_2+\cdots+x_m=n} \sum_{S \subseteq [m]} \prod_{i \in S} -[x_i \geq r_i] \\
&= \sum_{S \subseteq [m]} \sum_{x_1+x_2+\cdots+x_m=n} \prod_{i \in S} [x_i \geq r_i] \\
&= \sum_{S \subseteq [m]} (-1)^{|S|} \binom{n - \sum_{i \in S} r_i + m - 1}{m - 1}
\end{aligned}$$

接下来可以 2^m 枚举 S 来计算。

也可以枚举 $R = \sum_{i \in S} r_i$ 得到

$$\begin{aligned} & \sum_{S \subseteq [m]} (-1)^{|S|} \binom{n - \sum_{i \in S} r_i + m - 1}{m - 1} \\ &= \sum_R \binom{n - R + m - 1}{m - 1} \sum_{S \subseteq [m]} (-1)^{|S|} \left[\sum_{i \in S} r_i = R \right], \end{aligned}$$

而后者可以背包计算：

$$\begin{aligned} f_{m,R} &= \sum_{S \subseteq [m]} (-1)^{|S|} \left[\sum_{i \in S} r_i = R \right] \\ &= \sum_{S \subseteq [m-1]} (-1)^{|S|+1} \left[r_m + \sum_{i \in S} r_i = R \right] + \sum_{S \subseteq [m]} (-1)^{|S|} \left[\sum_{i \in S} r_i = R \right] \\ &= -f_{m-1, R-r_m} + f_{m-1, R}. \end{aligned}$$

n 个数的环排列个数

k 种颜色的球, 第 i 种颜色的有 n_i 个, 求方案数

P2476 [SCOI2008] 着色方案: 同上, 但是要求相邻颜色不相同

*P6151 [集训队作业 2019] 青春猪头少年不会梦到兔女郎学姐

P5339 [TJOI2019] 唱、跳、rap 和篮球

对于第一个问题, 每个环排列通过转圈对应到 n 个不同的排列, 所以方案数是 $n!/n = (n-1)!$ 。

对于第二个问题, 每个等价类通过对同色球内部的排列对应到 $\prod_i n_i!$ 个不同的排列, 所以方案数是 $\frac{n!}{\prod_i n_i!}$ 。或者, 也可以一个颜色一个颜色放置, 得到

$$\binom{n}{n_1} \binom{n-n_1}{n_2} \binom{n-n_1-n_2}{n_3} \cdots \binom{n-n_1-\cdots-n_{k-1}}{n_k} = \frac{n!}{\prod_i n_i!}.$$

也就是多项式系数 $\binom{n}{n_1, \dots, n_k}$

长为 n 的字符集大小为 m 的字符串，不能出现连续 k 个相同字符，求方案数

*P1393 Mivik 的标题：长为 n 的字符集大小为 m 的字符串，不能出现子串 S ，求方案数

P6076 [JSOI2015] 染色问题

P4491 [HAOI2018] 染色

第一个问题

做法一：设 $f_{i,j}$ 表示长为 i 的字符串，目前结尾极长连续段长度为 j ，且从来没有出现过连续 k 个相同字符的方案数，则 $f_{i,j} = f_{i-1,j-1} (j > 1)$ 以及 $f_{i,1} = \sum_{j < k} (m-1) f_{i-1,j}$ 。
 $O(nk)$ 直接做或者 $O(k^3 \log n)$ 矩阵加速。

做法二：设 f_i 表示长为 i 的字符串，且从来没有出现过连续 k 个相同字符的方案数。那么首先考虑 mf_{i-1} ，然后再减掉不合法的方案数。不合法的情况一定是最后 k 个字符都相同，且倒数第 $k+1$ 个字符和后面的不同。最后 k 个字符有 m 种可能；对于前面的串，由对称性，以每种字符结尾的合法串的个数都是 f_{i-k}/m ，所以不合法方案数总共是 $(m-1)f_{i-k}$ ，故 $f_i = mf_{i-1} - (m-1)f_{i-k}$ 。 $O(n)$ 直接做或者 $O(k^2 \log n)$ 甚至 $O(k \log k \log n)$ 线性递推。

做法三：容斥，考虑对于每个位置钦定以这个位置结尾有 k 个连续相同字符。然而剩下的事情十分困难，我们只给出生成函数的解释。想象一下钦定之后的序列会长成什么样子：序列被分成若干段，每一段要么是一个未被钦定的字符，要么是因为若干个间距小于 k 的钦定位置而连成的一个大同色连续段。将这个结构翻译成生成函数：前者是 mx ，后者是 $\frac{-mx^k}{1+\sum_{i=1}^{k-1}x^i}$ （注意每个钦定带来 -1 的系数）。最后再套一层序列就得到

$$\left(1 - mx - \frac{-mx^k}{1 + \sum_{i=1}^{k-1} x^i}\right)^{-1} = \frac{1 - x^k}{1 - mx + (m-1)x^k}.$$

做法四：考虑先枚举所有极长连续段的长度 $l_1, \dots, l_t$ ，其贡献为 $m(m-1)^{t-1}$ 。这样我们也可以直接写出生成函数

$$\frac{m}{m-1} \cdot \frac{1}{1 - (m-1)\sum_{i=1}^{k-1} x^i} = \frac{m}{m-1} \cdot \frac{1-x}{1 - mx + (m-1)x^k}$$

两个生成函数看起来有微妙的不同，这是 f_0 导致的。事实上，上面那个生成函数才是完全正确的，下面这个会导致 $f_0 = m/(m-1)$ 。这两个分式对应的线性递推和做法二是相同的。

另外，也可以直接展开生成函数：

$$\begin{aligned}
 [x^n] \frac{1-x^k}{1-mx+(m-1)x^k} &= [x^n] \frac{1}{1-mx} \cdot \frac{1-x^k}{1+\frac{(m-1)x^k}{1-mx}} \\
 &= [x^n] \frac{1-x^k}{1-mx} \sum_{i \geq 0} (-1)^i \left(\frac{(m-1)x^k}{1-mx} \right)^i \\
 &= \sum_{i \geq 0} (-1)^i (m-1)^i [x^n] \frac{(1-x^k)x^{ik}}{(1-mx)^{i+1}} \\
 &= \sum_{i \geq 0} (-1)^i (m-1)^i \left([x^{n-ik}] \frac{1}{(1-mx)^{i+1}} - [x^{n-(i+1)k}] \frac{1}{(1-mx)^{i+1}} \right) \\
 &= \sum_{i \geq 0} (-1)^i (m-1)^i \left(m^{n-ik} \binom{n-ik+i}{n-ik} - m^{n-(i+1)k} \binom{n-(i+1)k+i}{n-(i+1)k} \right)
 \end{aligned}$$

$i > n/k$ 后的项都是 0。每一个组合数都可以 $O(i) = O(n/k)$ 计算，总复杂度 $O((n/k)^2)$ 。

上面这种手法是常用的，即把分式写作 $\frac{1}{A(x)-B(x)} = \frac{1}{A(x)} \frac{1}{1-B(x)/A(x)}$ 然后展开。

第二个问题

前面的容斥做法可以扩展到一般的禁止子串 S , 只需要重新刻画由若干个间距小于 $k = |S|$ 的钦定位置而连成的一个大固定串。这样的东西形如 S 后面跟着若干个 S 的长度小于 k 的周期。所以总的生成函数就是

$$\left(1 - mx - \frac{-x^k}{1 + c(x)}\right)^{-1},$$

其中

$$c(x) = \sum_{i=1}^{k-1} [i \text{ 是 } S \text{ 的周期}] x^i.$$

从 $(0, 0)$ 走到 (n, m) , 只能向上向右走, 求方案数

从 $(0, 0)$ 走到 (n, m) , 只能向上向右走, 且任一时刻不能跨过直线 $y = x$, 求方案数

从 $(0, 0)$ 走到 (n, m) , 可以向上向右向右上走, 求方案数

把以上问题顺时针旋转 45°

从 $(0, 0)$ 走到 (n, m) , 只能向上向右走, 要求首次走到 $x = p$ 时所在的 y 坐标 $\leq q$

*Dyck 路: 从 $(0, 0)$ 走到 (n, m) , 不跨过直线 $y = kx + b$ 的方案数

第一个问题：由于必须恰好有 n 个向右的步和 m 的向上的步，任意安排其顺序，方案数是 $\binom{n+m}{n}$ 。这也展示了组合数的对称性。另外，如果我们枚举首次走到 $x = n$ 时所在的 y 坐标 i ，就可以得到上指标求和或者平行求和

$$\sum_{i=0}^m \binom{n-1+i}{i} = \binom{n+m}{m} = \binom{n+m}{n} = \sum_{i=0}^m \binom{n-1+i}{n-1}.$$

(当然也可以枚举首次走到 $y = m$ 时的 x 坐标)

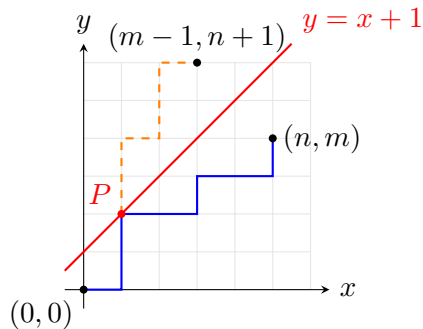
第三个问题：枚举向右上走的步数 i ，那么还需要 $n-i$ 个向右的步和 $m-i$ 个向上的步，任意安排其顺序，方案数是

$$\sum_i \binom{n+m-i}{i, n-i, m-i}$$

这两个问题的二元生成函数都非常好写，分别是 $\frac{1}{1-x-y}$ 和 $\frac{1}{1-x-y-xy}$ 。对其分析可以得到更多有用的结果。

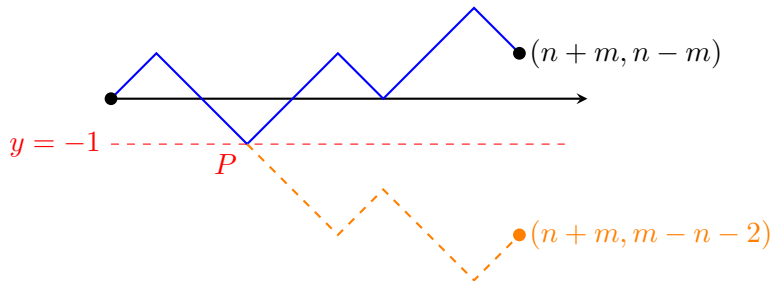
反射容斥

考虑第二个问题。假设 $m \leq n$ ，否则答案总是 0。我们用总的减去不合法的，而不合法的路径相当于是**碰到**直线 $y = x + 1$ 的路径。对于这样一条路径，我们把路径中**首次**碰到直线之后的部分沿着 $y = x + 1$ 对称，这样我们就得到了一条从 $(0, 0)$ 到 $(m - 1, n + 1)$ 的路径。反过来，考虑任何一条从 $(0, 0)$ 到 $(m - 1, n + 1)$ 的路径，这样的路径必然穿过直线 $y = x + 1$ 。做同样的对称操作，我们就得到了一条从 $(0, 0)$ 走到 (n, m) 且碰到 $y = x + 1$ 的路径。也就是说这种变换是一个双射，从而不合法的路径数量就等于从 $(0, 0)$ 走到 $(m - 1, n + 1)$ 的路径数量，即 $\binom{n+m}{m-1}$ 。



反射容斥：旋转 45°

在顺时针旋转 45° 再关于 x 轴镜像后的图上想象这个过程或许会更容易。此时，不合法的路径是不碰到直线 $y = -1$ 的路径。



另外，这个容斥也可以推广到不碰到两条线的情况。

在前面的问题中考虑 $n = m$ 的特殊情况，此时方案数是

$$C_n = \binom{2n}{n} - \binom{2n}{n-1} = \frac{1}{2n+1} \binom{2n+1}{n}.$$

这个数列被称为**卡特兰数** (Catalan Number)，它也在很多其他问题中自然地出现：

- 长为 $2n$ 的合法括号序列数量
- n 个 1 和 n 个 -1 组成的满足所有前缀和都非负的序列数量
- n 个点的儿子有序的二叉树数量
- $n+1$ 个点的儿子有序的多叉树数量
- n 个元素的出栈序数量
- $n+2$ 边形的三角剖分数

括号序列

一个括号序列长什么样子？我们考虑所有最外层的括号，就得到 $\mathcal{A} = (\mathcal{A})(\mathcal{A}) \cdots (\mathcal{A})$ 。或者，我们可以把除了第一个括号以外的看成一体，就得到 $\mathcal{A} = \{\epsilon\} + (\mathcal{A})\mathcal{A}$ (ϵ 代表空序列)。把这两个构造翻译成递推式就得到

$$C_n = \sum_{k \geq 0} \sum_{(i_1+1) + \cdots + (i_k+1) = n} C_{i_1} \cdots C_{i_k}, \quad C_n = [n=0] + \sum_{i+1+j=n} C_i C_j.$$

更好的做法是翻译成生成函数：

$$C(x) = \frac{1}{1 - xC(x)}, \quad C(x) = 1 + xC(x)^2.$$

也可以翻译成区间 dp，比如 [CSP-S 2021] 括号序列（不过这个题稍微复杂一点点）。

括号序列与 ± 1 序列

我们把 (看成 $+1$,) 看成 -1 , 那么括号序列就一一对应由 n 个 $+1$ 和 n 个 -1 组成的序列。这个转换是非常常用的。

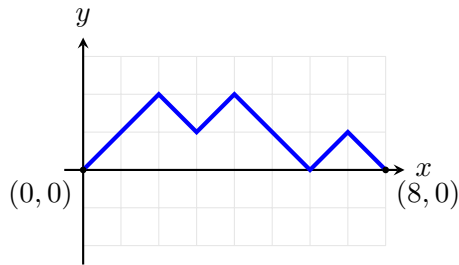
定理

括号序列合法当且仅当对应的 ± 1 序列的任何前缀和都非负。

这样我们就建立了这两个组合类之间的双射。

数列与格路径

另一方面，我们也经常把数列按照如下方式转化成格路径：从 $(0, 0)$ 出发，逐个考虑 $i = 1, \dots, n$ ，遇到 a_i 就走一步向量 $(1, a_i)$ 。例如， $+1, +1, -1, +1, -1, -1, +1, -1$ 对应下图所示的路径：



记 s_i 为数列的前缀和，则容易发现 $x = i$ 处的高度为 s_i 。这也说明了图上两个点 (i, s_i) 和 (j, s_j) 之间的高度差对应数列的区间和 $s_j - s_i = \sum_{j < k \leq i} a_k$ 。

所有前缀和都非负对应着不触碰 $y = -1$ 。这样，我们就建立起了 ± 1 序列与先前讨论的格路径之间的双射。

通过前面两个双射，我们证明了括号序列与 ± 1 序列的数量都是卡特兰数，所以先前对括号序列建立的递推式也就对应着卡特兰数的递推式。

然而不同的组合结构带给人的直观感受是不同的！对于 ± 1 序列，我们几乎没有任何直观；对于括号序列，我们有序列分解的直观；对于格路径，我们有图像的直观。另外，前一页中的格路径似乎比之前旋转 45° 之前的格路径更加直观。

事实上，我们也很容易对格路径建立先前对括号序列建立的递推：只需要考虑每一次触碰 $y = 0$ 而分成的若干段即可。

我们还有另一个方法来得到卡特兰数的封闭形式，这需要 Raney 引理：

引理 (Raney 引理)

对于一个和为 1 的整数序列，其所有循环移位中恰好有一个满足：所有前缀和都是正的

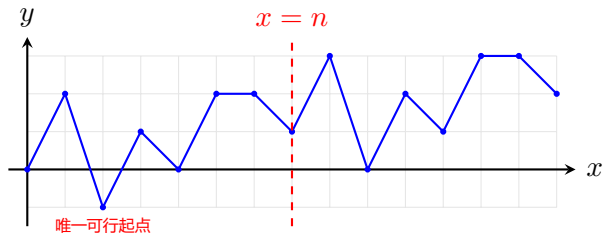
它有一个（我们暂时用不到的）推广如下：

引理 (推广的 Raney 引理)

对于一个和为 ℓ 且所有元素都 ≤ 1 的整数序列，其所有循环移位中恰好有 ℓ 个满足：所有前缀和都是正的

证明

循环移位可以通过把序列复制一遍来处理。也就是说如果我们令 $a_{n+i} = a_i$ ，那么循环移位 t 后得到的序列就是 $a[t+1, n+t]$ 。我们现在可以把复制一遍的序列画成格路径：



现在, $[i+1, n+i]$ 这一段区间对应的格路径相当于以 (i, s_i) 为起点、以 $(n+i, s_{n+i})$ 为终点的格路径, 而所有前缀和均为正相当于过程中高度始终高于起点。那么显然有且仅有以那个最小的 s_i (相等的话认为 i 大的小) 为起点才能做到这一点。

± 1 序列：再探

我们需求的是所有前缀和非负，而 Raney 引理中是所有前缀和 > 0 。我们可以通过在序列开头加一个 1 来处理这个问题。也就是说我们考虑双射： n 个 $+1$ 和 n 个 -1 组成的所有前缀和非负的序列 $\leftrightarrow$ 由 $n+1$ 个 $+1$ 和 n 个 -1 组成的所有前缀和 > 0 的序列。

对于新的问题，Raney 引理告诉我们可以将所有的 $\binom{2n+1}{n}$ 个序列按照循环移位划分等价类，而每个等价类恰好贡献一种合法方案。由于 $\gcd(n+1, n) = 1$ ，每个等价类的大小都是 $2n+1$ （也就是说一个序列的所有循环移位都是不同的），除法原理就告诉我们合法方案数为

$$\frac{1}{2n+1} \binom{2n+1}{n}.$$

二叉树

二叉树具有自然的左子树-根-右子树结构, 也就是 $\mathcal{A} = \{\epsilon\} + \begin{array}{c} \bullet \\ / \quad \backslash \\ \mathcal{A} \quad \mathcal{A} \end{array}$, 所以二叉树和合法括号序列有相同的递推式。又因为初值也相同, 所以二叉树的数量也是卡特兰数。

我们也可以把这个结构与括号序列的结构进行对应, 据此得到一个双射: 记 $S[T]$ 为树 T 映射到的括号序列, 则

$$S \left[\begin{array}{c} \bullet \\ / \quad \backslash \\ L \quad R \end{array} \right] = (S[L])S[R], \quad S[\epsilon] = \epsilon.$$

另外, 我们还需要注意 n 个点的二叉树与有 n 个**内点**的满二叉树³的一一对应 (内点即左右子树都非空的节点): 想象在每个非内点下面长叶子来变成内点

³这里满二叉树指所有非叶结点都有两个儿子的二叉树

通过枚举儿子数量，我们得到

$$\mathcal{A} = \bullet + \begin{array}{c} \bullet \\ | \\ \mathcal{A} \end{array} + \begin{array}{c} \bullet \\ / \quad \backslash \\ \mathcal{A} \quad \mathcal{A} \end{array} + \begin{array}{c} \bullet \\ / \quad | \quad \backslash \\ \mathcal{A} \quad \mathcal{A} \quad \mathcal{A} \end{array} + \cdots$$

那么具有 $n + 1$ 个点的多叉树和长为 $2n$ 的括号序列有相同的递推式。我们同样可以据此得到一个双射：

$$S \left[\begin{array}{c} \bullet \\ / \quad \cdots \quad \backslash \\ T_1 \quad \cdots \quad T_k \end{array} \right] = (S[T_1]) \cdots (S[T_k])$$

出栈序

维护一个栈 S 和一个整数 $i = 1$, 在任意时刻可以 (i) 如果 $i \leq n$, 则将 i 入栈, 并令 $i \leftarrow i + 1$ 或者 (ii) 如果栈非空, 则弹出当前栈顶。等到无法继续操作时, 按照出栈的顺序可以得到一个排列。这就是 $1, \dots, n$ 的出栈序。这是一个不太常用的模型, 因为它不够直观而且分析比较复杂。

考虑最后一次操作, 这一定是一个出栈。假设这次出栈的元素是 x , 那么考虑 x 的入栈时刻 t , 则在 t 时刻结束后栈中一定只有 x 自己 (否则 x 无法最后出栈)。因此, 出栈序可以按顺序划分成三部分: $[1, x]$ 的出栈序、 $(x, n]$ 的出栈序、 x 。这又是一个卡特兰数类型的递归。

另一方面, 我们可以注意到操作序列与出栈顺序的联系。由于每个元素出入栈各一次, 序列的长度必然是 $2n$ 。因此, 上面的分析也给出了操作序列的划分, 于是出栈序与操作序列一一对应。而操作序列天然对应一个括号序列 (入栈/出栈分别对应左/右括号)。

按照前面的分析，我们还可以给出和二叉树的一个双射：我们给二叉树按照中序遍历编号，则出栈序对应一个后序遍历。我们知道中序遍历 + 先/后序遍历可以唯一确定一棵二叉树。

最后，出栈序等价于 312 禁子排列，即不存在 $i < j < k$ 但是 $\pi_j < \pi_k < \pi_i$ 这种模式的排列。因为如果我们考虑 π_n ，则 $k = n$ 的限制的影响是 $(\pi_n, n]$ 必须出现在 $[1, \pi_n)$ 右边，然后剩下的限制只有这两块内部的限制。

多边形三角剖分

顺时针给所有顶点编号。考虑最终剖分中 $1 - n$ 这条边在其所在的三角形中的对点 i ，则此三角形划分出了两个子结构，规模分别为 i 和 $n - i + 1$ 。这是 C_{n+2} 满足的递归。

可以据此设计到二叉树的双射：每个三角形视为一个点，与相邻的三角形连边。事实上，这就是平面图转对偶图。

组合结构往往具有自然的子结构。最常见的包括：

- 序列：前缀/区间
- 树：子树/链
- 集合：子集
- 网格：1/2-side 子矩形；轮廓线
- DAG：子 DAG（拓扑序）

[CSP-S 2021] 括号序列

合适地刻画子问题，使得每种子问题对后续状态的转移系数都相同。

例：[CSP-S 2025] 员工招聘

做法零：将 (前 i 个人, 要求形成的部分排列为 P , 的部分排列数量) 作为子问题。 $\tilde{O}(n!)$

做法一：将 (前 i 个人, 要求构成的集合为 S 、目前未录用的人数为 j , 的部分排列数) 作为子问题。 $\tilde{O}(2^n)$ 。这个刻画对大部分排列类计数都适用。

做法二 (伪)：将 (前 i 个人, 要求目前未录用的人数为 j 、 $c > j$ 的人中还没放的人数为 k , 的部分排列数) 作为子问题。然而, 这样不能转移, 因为无法知道下一个位置不录用时再下一个位置的候选人数会如何变化。

做法二 (真)：上一个做法中, “无法知道下一个位置不录用时再下一个位置的候选人数会如何变化” 的原因如下: 假设目前不录用人数是 j , 而之前可能已经录用了一些 $c = j + 1$ 的人, 但这个人数没有在状态中记录。所以我们可以考虑把 $c = j + 1$ 的人等到最后再一起放了。也就是将 (前 i 个人, 要求目前未录用的人数为 j 、 $c > j$ 的人中还没放的人数为 k 、只确定了 $c \leq j$ 的人的位置, 形成的部分排列数) 作为子问题。这个技巧也称为 “贡献延后计算”。

另一个视角

假设我们先钦定了所有位置的录取情况，那么如何计算方案数？对于一个录取的位置，必须有 $s_i = 1$ 且 $c_{\pi_i} > f_i$ ，其中 f_i 为 i 前面的未录取人数；对于一个未录取的位置，需要 $s_i = 0$ 或者 $c_{\pi_i} \leq f_i$ 。那么相当于每个位置有限制 $c_{\pi_i} > f_i$ 或者 $c_{\pi_i} \leq f_i$ 或者无限制。

现在来考虑子问题：(i) 如果只有某一种限制怎么做？(ii) 如果只有某两种限制怎么做？(iii) 怎么把某一种限制用另外两种限制表示？

如果把无限制表示成 $[c_{\pi_i} > f_i] + [c_{\pi_i} \leq f_i]$ ，那就能得到上一页中的做法。如果把 $[c_{\pi_i} > f_i]$ 表示成 $1 - [c_{\pi_i} \leq f_i]$ ，那么就得到另一种做法，也就是容斥。

最后，可以在 dp 中枚举所有位置的录取情况。

生成函数初步

这里主要介绍怎么用生成函数 (Generating Function, GF) 处理序列 (假设已经得到序列的某些性质, 如递推关系等)。事实上更多情况下可以直接对着问题的结构写出生成函数, 而无需额外经过一步 dp 转化。绝大多数的推式子问题都有办法用生成函数处理。但需要强调的是生成函数**并非万能**, 不应忽视其他的技巧和组合意义。在使用生成函数之前先做一些基本的处理往往能简化推导。

最基本和主要的手段就是把需要处理的序列 (f_i) 挂到幂级数 $\sum_{i \geq 0} f_i a_i x^i$ 上去, 其中 a_i 是自己选定的另一组系数, 通常是 1 (普通生成函数, Ordinary GF) 或 $1/i!$ (指数生成函数, Exponential GF)。这样导出的幂级数几乎总是比序列容易处理的。注意在实践中一般需要假设 $x \approx 0$ 是一个小量。我们后面说的“幂级数”和“生成函数”都是指的同一个东西。

记 $[x^n]F(x)$ 为幂级数 $F(x)$ 的第 n 次项系数。在上下文清楚时, 有时我们也用 f_n 代表这个事情。(注意: 如果上面挂到幂级数上时使用了非 1 的 a_i , 那么这里的 f_n 代表上面 $\sum_{n \geq 0} f_n a_n x^n$ 中的 $f_n a_n$ 而不是 f_n 本身, 后同)

可以对幂级数与它的封闭形式做同样的运算来得到以下的对应关系：

线性运算 $H(x) = aF(x) + bG(x)$ 对应 $h_n = af_n + bg_n$

卷积 $H(x) = F(x)G(x)$ 对应 $h_n = \sum_{i=0}^n f_i g_{n-i}$ 。更一般地，

$$[x^n]F_1(x)F_2(x)\cdots F_m(x) = \sum_{i_1+i_2+\cdots+i_m=n} ([x^{i_1}]F_1(x))([x^{i_2}]F_2(x))\cdots([x^{i_m}]F_m(x)).$$

求导 $H(x) = F'(x)$ 对应 $h_n = (n+1)f_{n+1}$ 。

导数有几个不同的记号, $f'(x)$ 、 $\frac{d}{dx}f(x)$ 、 $\mathbf{D}f(x)$ 都可以用来代表求导。如果需要反复求导 (高阶导数), 那么可以用 $f'''(x)$ 、 $f^{(k)}(x)$ 、 $\frac{d^k}{dx^k}f(x)$ 、 $\mathbf{D}^k f(x)$ 这些记号。

我们做的所有操作都是**形式上的**。你只需要 (熟练) 记住以下这些事实:

- 运算性质 $(af(x) + bg(x))' = af'(x) + bg'(x)$, $(f(x)g(x))' = f'(x)g(x) + f(x)g'(x)$,
 $(f(g(x)))' = f'(g(x))g'(x)$
- 初等函数的导数 $(x^\alpha)' = \alpha x^{\alpha-1}$, $(\ln x)' = x^{-1}$, $(e^x)' = e^x$

下面是几个练习。这些结果也是有价值且值得记住的:

- 常数的导数是 0
- $(f(x)/g(x))' = \frac{f'(x)g(x) - f(x)g'(x)}{g(x)^2}$
- $(\log f(x))' = \frac{f'(x)}{f(x)}$
- $(e^{f(x)})' = f'(x)e^{f(x)}$
- $(fg)^{(k)} = \sum_i \binom{k}{i} f^{(i)} g^{(k-i)}$

运算法则 II

$H(x) = x^m F(x)$ 对应移位 $h_n = f_{n-m}$ 。注意：尽量不要让你的式子中出现**负下标项**，所以在 m 为负时要小心。。

$H(x) = \frac{1}{1-x} F(x)$ 对应前缀和 $h_n = \sum_{i \leq n} f_i$

$H(x) = (1-x)F(x)$ 对应差分 $h_n = f_n - f_{n-1}$

$H(x) = \vartheta F(x) \stackrel{\text{def}}{=} xF'(x)$ 对应原地的求导 $h_n = n f_n$

$H(x) = F(x) \bmod (x^k - 1)$ 对应 $h_r = \sum_n f_{kn+r}$ 。（事实上模一般的 $Q(x)$ 相当于按一个线性递推累加系数）

$H(x) = x^k F^{(k)}(x)$ 对应 $h_n = n^k f_n$

$H(x) = (x\mathbf{D})^k F(x)$ 对应 $h_n = n^k f_n$ 。然而， $(x\mathbf{D}x)^k$ 一般没有好的封闭形式（虽然 $(x\mathbf{D})^k = \sum_i \left\{ \begin{smallmatrix} k \\ i \end{smallmatrix} \right\} x^i \mathbf{D}^i$ ）

最重要的一些生成函数

下面几个恒等式是应该（双向）记住的，因为它们实在是太常见了。

$$(1+x)^\alpha = \sum_{n \geq 0} \binom{\alpha}{n} x^n, \quad \ln \left(\frac{1}{1-x} \right) = \sum_{n \geq 1} \frac{x^n}{n} \quad e^x = \sum_{n \geq 0} \frac{x^n}{n!}$$

第一个恒等式——也就是二项式定理——的一些特殊情况也是值得记忆的（证明它们!）：

$$\frac{1}{(1-x)^k} = \sum_{n \geq 0} \binom{n+k-1}{n} x^n, \quad \frac{x^k}{(1-x)^{k+1}} = \sum_{n \geq 0} \binom{n}{k} x^n, \quad \frac{1}{\sqrt{1-4x}} = \sum_{n \geq 0} \binom{2n}{n} x^n$$

x 可以代入为任何 ≈ 0 的东西 (有些时候还可以代入别的东西⁴, 取决于**收敛半径**)。在生成函数的语境下, ≈ 0 的东西指的就是任何**常数项为 0** 的幂级数。

直接的例子:

$$F(x^k) = \sum_{n \geq 0} f_n x^{nk}, \quad F(rx) = \sum_{n \geq 0} f_n r^n x^n.$$

复杂一点的例子:

$$\sum_{n \geq 0} \left(\frac{x}{1-x} \right)^n = \frac{1}{1 - \frac{x}{1-x}} = \frac{1-2x}{1-x}$$

代值的例子: 把 $x = 1$ 和 $x = -1$ 分别代入 $(1+x)^n$ 。把 $|r| < 1$ 的 r 代入 $1/(1-x)^k$ 。

⁴在 OI 里, 你基本可以认为是任何东西

使用例

化简范德蒙德卷积 $\sum_i \binom{r}{i} \binom{s}{n-i}$ 。我们只需要乘上 x^n 然后对 n 求和，化简得到的生成函数，最后再提取 $[x^n]$ 。也就是

$$[x^n] \sum_n x^n \binom{r}{i} \binom{s}{n-i}.$$

用这个方法化简（注意：你需要决定对哪个变量做我们前面说的事情）

$$\sum_k \binom{l}{m+k} \binom{s+k}{n} (-1)^k, \quad \sum_{-q \leq k \leq l} \binom{l-k}{m} \binom{q+k}{n}, \quad \sum_{k \geq 0} \binom{n+k}{m+2k} \binom{2k}{k} \frac{(-1)^k}{k+1}.$$

（然而问题 8 没有办法直接这样做... 但是有另外的技巧可以做...）

试着对下面的式子做同样的事情（我们一会再来教大家怎么提取这种系数）

$$\sum_{k \leq n} \binom{n-k}{k} (-1)^k$$

对斐波那契数列的递推公式 $f_n = f_{n-1} + f_{n-2}$ 使用前面的方法。根据得到的方程解出生成函数。通过对生成函数求导，导出一个 $\sum_{i=0}^n f_i f_{n-i}$ 的简化形式。

对卡特兰数的递推公式 $C_n = \sum_{i=0}^{n-1} C_i C_{n-i-1}$ 使用前面的方法。据此导出卡特兰数的封闭形式。

对错排的递推公式 $d_n = (n-1)(d_{n-1} + d_{n-2})$ 、 $d_n = nd_{n-1} + (-1)^n$ 使用前面的方法（试试 OGF 和 EGF，也就是分别乘 x^n 和 $x^n/n!$ ）。

我们看看如何用生成函数处理以前的一些问题。这里，我们可以先只关心生成函数的封闭形式，等会再考虑如何计算。

$$\sum_{0 \leq i \leq n} \text{Fib}_i, \quad \sum_{1 \leq i_1 \leq i_2 \leq \dots \leq i_m \leq n} 1, \quad \sum_{i=0}^n r^i i^k, \quad \sum_i \binom{n}{i} r^i a_{i \bmod k}.$$

$$\sum_i \binom{n}{i} r^{\lfloor i/k \rfloor}$$

(假设 $r^{1/k}$ 和 ω_k 都不存在)。感觉是不太广为人知的东西。

可以使用一些深刻的背景知识将其扩展到 $\langle H(x), F(x) \rangle$, 其中 H 是某个稀疏多项式, $F(x)$ 是任意线性递推, $\langle \cdot, \cdot \rangle$ 表示系数点积。

得到一个生成函数的封闭形式后，如何用于计算第 n 项或者前 n 项？下面是几种常见的方法：

- 对于比较简单的情况，可以直接使用前面的常见级数展开来得到系数。
 - 这里可能直接得到封闭形式，也可能得到一个卷积求和
- 对于有理分式，可以分式分解，或者使用线性递推
- 使用 FFT 来进行各种多项式运算
- 使用暴力来进行各种多项式运算
- 通过求导等手段得到递推式
- 拉格朗日反演

分式分解

对于 $P(x)/Q(x)$ (P, Q 都是多项式) 的形式, 可以先将其变成 $P(x) \operatorname{div} Q(x) + \frac{P(x) \bmod Q(x)}{Q(x)}$ 。然后可以假设 $\deg P < \deg Q$ 。(有人不会多项式除法吗?)

真分式总是有如下的分解:

$$\frac{P(x)}{\prod (a_i x + b_i)^{k_i}} = \sum_i \sum_{j=1}^{k_i} \frac{c_{ij}}{(a_i x + b_i)^j}.$$

可以待定系数计算 c_{ij} 。当 $k_i = 1$ 时, 有更方便的算法。两侧同乘 $a_i x + b_i$, 然后代入 $x_i = -b_i/a_i$ 就得到

$$\frac{P(x_i)}{\prod_{j \neq i} (a_j x_i + b_j)^{k_j}} = c_{i1}$$

对于有理真分式 $F(x) = P(x)/Q(x)$, 两侧乘 $Q(x)$ 得到 $F(x)Q(x) = P(x)$ 。展开卷积就得到线性递推。

使用分式分解即可得到所有线性递推的相关结论。例如, 你现在应该能够自行推出

$$f_n = \sum_{i=1}^k a_i f_{n-i} + r^n \left(\sum_{i=0}^m b_i n^i \right)$$

的生成函数的封闭形式。

我们现在有非常简单的 $O(M(m) \log n)$ 算法来得到一个有理分式的第 n 次项系数: 注意到 $Q(x)Q(-x)$ 只有偶数项有值, 考虑 $[x^n] \frac{P(x)}{Q(x)} = [x^n] \frac{P(x)Q(-x)}{Q(x)Q(-x)}$, 后者只与分子的奇数/偶数次项有关 (取决于 $n \bmod 2$), 据此我们可以将 n 减半, 而分式的规模保持不变。

下面的 n, m 都指多项式的**项数**而不是次数。对 $F(x)$ 用 n , 对 $G(x)$ 用 m 。

- 可以在线性时间内计算 $aF(x) + bG(x)$ 、 $x^m F(x)$ 、 $F'(x)$ 、 $\int_0^x F(t)dt$ 。
- 可以在 $O(nm)$ 时间内计算 $F(x)G(x)$ 。所以当 $\min(n, m)$ 非常小时复杂度优秀
- 可以在 $O(m(\deg F - \deg G))$ 时间内计算 $F(x) \operatorname{div} G(x)$ 和 $F(x) \operatorname{mod} G(x)$
- 后两者可以用 FFT 做到 $O(N \log N)$, 其中 $N = \max(\deg F, \deg G)$

$O(n^2)$ 多项式运算 II

假设 F 的**项数** $< m$, 那么可以在 $O(nm)$ 时间内计算下列级数的前 n 项 (可以假设 $m \leq n$, 为什么?). 实践中 m 经常很小, 此时这种暴力的复杂度非常优秀。

- $G = F^{-1}H$ (需求 $f_0 \neq 0$): 也就是 $G \cdot F = H$, 直接展开卷积就得到一个递推式
- $G = \ln F$ (需要 $f_0 = 1$): 两侧求导得到 $G' = F'/F$, 也就是 $FG' = F'$, 展开卷积就得到一个递推式。初值 $g_0 = 0$ 。
- $G = e^F$ (需要 $f_0 = 0$): 两侧求导得到 $G' = F'G$, 展开卷积得到递推式。初值 $g_0 = 1$ 。
- $G = F^k$: 两侧求导得到 $G' = kF'G/F$, 也就是 $FG' = kF'G$, 展开卷积得到递推式。初值 $g_0 = f_0^k$ 。
- 练习: 假设 F 是分子分母项数都不超过 m 的有理分式, 在 $O(nm)$ 时间内求出 F 的前 n 项。

以上都可以用 FFT 做到 $O(n \log n)$ 。注意到在 m 为常数时, 这还不如上面的做法。

还有值得再讲一个专题的例子和技巧，然而这里时间不够。